VOL. 11

Blockchain (Solidity)

Smart contracts, tokens and DApps on Ethereum and beyond

Source	cryptozombies.io
Generated	2026-10-10
Level	Intermediate
Estimated study time	30 hours
Language	English
Volume	Vol. 11 - Blockchain

This volume is personal study material. The notes, examples, exercises and tests were written from scratch while following the source site's learning path. No content is affiliated with, endorsed by, or copied from the source site or the technology's owner.

Table of contents

How to use this volume 7

Chapter map 7

Chapter 1 - Blockchain and Ethereum fundamentals 8

1.1 Summary 8

1.2 Concepts 8

1.3 Examples 9

Example 1 - Reading the chain with JSON-RPC 9

Example 2 - Anatomy of a transfer 10

1.4 Common mistakes and best practices 10

1.5 Practice exercises 10

Solutions 11

1.6 Mini test 11

Answer key and explanations 11

Chapter 2 - Your first smart contract 11

2.1 Summary 11

2.2 Concepts 12

2.3 Examples 13

Example 1 - A counter contract 13

Example 2 - Overflow protection 13

2.4 Common mistakes and best practices 14

2.5 Practice exercises 14

Solutions 14

2.6 Mini test 15

Answer key and explanations 15

Chapter 3 - Data structures: structs and arrays 16

3.1 Summary 16

3.2 Concepts 16

3.3 Examples 17

Example 1 - A struct-based registry 17

Example 2 - Mappings instead of arrays 19

3.4 Common mistakes and best practices 19

3.5 Practice exercises 20

Solutions . . . 20
3.6 Mini test . . . 20
Answer key and explanations . . . 21
Chapter 4 - Functions, visibility and events . . . 21
4.1 Summary . . . 21
4.2 Concepts . . . 21
4.3 Examples . . . 22
Example 1 - Events and visibility in a registry . . . 22
Example 2 - Listening to events from the front-end . . . 24
4.4 Common mistakes and best practices . . . 24
4.5 Practice exercises . . . 24
Solutions . . . 25
4.6 Mini test . . . 25
Answer key and explanations . . . 25
Chapter 5 - Ownership, mappings and security basics . . . 26
5.1 Summary . . . 26
5.2 Concepts . . . 26
5.3 Examples . . . 27
Example 1 - Ownable plus a safe vault . . . 27
Example 2 - The vulnerable version, for contrast . . . 29
5.4 Common mistakes and best practices . . . 29
5.5 Practice exercises . . . 29
Solutions . . . 30
5.6 Mini test . . . 30
Answer key and explanations . . . 30
Chapter 6 - Inheritance, interfaces and imports . . . 31
6.1 Summary . . . 31
6.2 Concepts . . . 31
6.3 Examples . . . 32
Example 1 - Overriding and constructors . . . 32
Example 2 - Interfaces for calling other contracts . . . 33
6.4 Common mistakes and best practices . . . 34
6.5 Practice exercises . . . 34
Solutions . . . 35

6.6 Mini test . . . 35
Answer key and explanations . . . 35
Chapter 7 - Tokens and the ERC721 standard . . . 36
7.1 Summary . . . 36
7.2 Concepts . . . 36
7.3 Examples . . . 37
Example 1 - Minting NFTs with OpenZeppelin . . . 37
Example 2 - Reading token data from a front-end . . . 39
7.4 Common mistakes and best practices . . . 39
7.5 Practice exercises . . . 39
Solutions . . . 40
7.6 Mini test . . . 40
Answer key and explanations . . . 40
Chapter 8 - A payable game economy . . . 40
8.1 Summary . . . 40
8.2 Concepts . . . 41
8.3 Examples . . . 42
Example 1 - A cooldown-gated action with a fee . . . 42
Example 2 - Commit-reveal randomness (sketch) . . . 44
8.4 Common mistakes and best practices . . . 44
8.5 Practice exercises . . . 45
Solutions . . . 45
8.6 Mini test . . . 46
Answer key and explanations . . . 46
Chapter 9 - Front-ends and Web3.js . . . 46
9.1 Summary . . . 46
9.2 Concepts . . . 47
9.3 Examples . . . 47
Example 1 - Connecting a wallet and reading state . . . 47
Example 2 - Sending a transaction and watching events . . . 48
9.4 Common mistakes and best practices . . . 49
9.5 Practice exercises . . . 49
Solutions . . . 50
9.6 Mini test . . . 50

Answer key and explanations . . . 50
Chapter 10 - Tooling: Truffle, Ganache and deployment . . . 50
10.1 Summary . . . 50
10.2 Concepts . . . 51
10.3 Examples . . . 51
Example 1 - A full local workflow . . . 51
Example 2 - Deploying to a public testnet . . . 53
10.4 Common mistakes and best practices . . . 54
10.5 Practice exercises . . . 54
Solutions . . . 54
10.6 Mini test . . . 55
Answer key and explanations . . . 55
Chapter 11 - Testing smart contracts . . . 55
11.1 Summary . . . 55
11.2 Concepts . . . 56
11.3 Examples . . . 57
Example 1 - A focused test suite . . . 57
Example 2 - Testing the NFT events . . . 58
11.4 Common mistakes and best practices . . . 59
11.5 Practice exercises . . . 59
Solutions . . . 59
11.6 Mini test . . . 60
Answer key and explanations . . . 60
Chapter 12 - Oracles and external data . . . 60
12.1 Summary . . . 60
12.2 Concepts . . . 61
12.3 Examples . . . 62
Example 1 - Reading a Chainlink price feed . . . 62
Example 2 - A minimal request-fulfilment oracle . . . 63
12.4 Common mistakes and best practices . . . 63
12.5 Practice exercises . . . 64
Solutions . . . 64
12.6 Mini test . . . 64
Answer key and explanations . . . 64

Chapter 13 - The L2 and multi-chain landscape . . . 65
13.1 Summary . . . 65
13.2 Concepts . . . 65
13.3 Examples . . . 66
Example 1 - Chain-aware configuration in one file . . . 66
Example 2 - Detecting the chain in a contract . . . 66
13.4 Common mistakes and best practices . . . 67
13.5 Practice exercises . . . 67
Solutions . . . 68
13.6 Mini test . . . 68
Answer key and explanations . . . 68
Chapter 14 - Capstone: critter-battle DApp . . . 68
14.1 Summary . . . 68
14.2 Concepts . . . 69
14.3 Examples . . . 69
Example 1 - Battle settlement (the heart of the DApp) . . . 69
Example 2 - Front-end flow for the capstone . . . 71
Example 3 - Release checklist for the capstone . . . 71
14.4 Common mistakes and best practices . . . 72
14.5 Practice exercises . . . 72
Solutions . . . 72
14.6 Mini test . . . 73
Answer key and explanations . . . 73
Cheat sheet . . . 73
15.1 Language basics . . . 73
15.2 Visibility and mutability . . . 74
15.3 Units and constants . . . 74
15.4 Gas quick reference . . . 74
15.5 Battle-hardened snippets . . . 75
15.6 Tooling commands . . . 75
15.7 Front-end essentials . . . 75
Final test . . . 75
16.1 Final test answers . . . 76
Sources . . . 77

How to use this volume

Prerequisites. Vol. 01 (HTML), Vol. 02 (CSS) and Vol. 03 (JavaScript). No prior blockchain experience is assumed: the book starts from accounts, gas and transactions.

Estimated study time. 30 hours: roughly 2 hours per chapter for reading, typing the examples and doing the exercises.

How the chapters work. Chapters 1 to 3 build the mental model and the language. Chapters 4 to 8 cover the Solidity patterns used in every real project: visibility, events, ownership, inheritance, tokens and payments. Chapters 9 to 13 add the engineering around the contract: front-ends, tooling, tests, oracles and other networks. Chapter 14 integrates everything.

How to run the code. Solidity contracts run on a blockchain, so the cheapest way to practise is the Remix browser IDE for the contract examples and a local Ganache node for the command-line chapters. The commands below check your toolchain before chapter 10:

shell

```
node --version
npm --version
npx truffle version
```

Expected output (versions on your machine will differ):

output

```
v22.21.0
10.9.4
Truffle v5.11.5 (core: 5.11.5)
Solidity v0.8.20 (solc-js)
Node v22.21.0
Web3.js v4.4.0
```

Note on examples. Code is written for Solidity `^0.8.24` with OpenZeppelin `5.x`. Contracts that import packages assume `npm install @openzeppelin/contracts @chainlink/contracts` ran inside the project. Every example shows its expected result as a text block.

Chapter map

#	Chapter	You will learn
1	Blockchain and Ethereum fundamentals	blocks, nodes, consensus, accounts, gas
2	Your first smart contract	pragma, state variables, arithmetic
3	Data structures: structs and arrays	structs, arrays, mappings, storage vs memory
4	Functions, visibility and events	external, internal, view, pure, events
5	Ownership, mappings and security basics	msg.sender, require, reentrancy-safe payments
6	Inheritance, interfaces and imports	virtual, override, abstract, OpenZeppelin

#	Chapter	You will learn
7	Tokens and the ERC721 standard	ERC20 vs ERC721, minting, metadata
8	A payable game economy	msg.value, cooldowns, safe randomness
9	Front-ends and Web3.js	MetaMask, contract calls, events in the UI
10	Tooling: Truffle, Ganache and deployment	migrate, deploy, verify
11	Testing smart contracts	Mocha, Chai, actions, event assertions
12	Oracles and external data	request/receive, Chainlink price feeds
13	The L2 and multi-chain landscape	rollups, zkSync, other EVM chains
14	Capstone: critter-battle DApp	contract + tests + front-end together

Chapter 1 - Blockchain and Ethereum fundamentals

1.1 Summary

- A blockchain is a chain of blocks in which every block stores the hash of its predecessor, so rewriting history means recomputing every later hash.
- A network of independent nodes each keeps a full copy of the ledger and agrees on which block comes next through a consensus protocol.
- Ethereum currently uses proof of stake: validators propose and vote on blocks, and misbehaviour is punished economically.
- Ether (ETH) exists to pay for computation: every operation has a gas price, and the sender pays for the gas its transaction consumes.
- There are two kinds of accounts: externally owned accounts controlled by keys, and contract accounts controlled by code.
- A transaction is a signed message that either moves ether, deploys code, or calls a function on a contract.
- A smart contract is a program that lives at an address and runs deterministically on every node in the network.
- Because execution is deterministic, contracts cannot make HTTP calls or read the wall clock: they can only use data already on chain.

1.2 Concepts

Each block contains a list of transactions and the hash of the previous block. Change one byte in block 100 and every later hash changes, so the network rejects the branch:

output

```
block 98        block 99        block 100
+-----------+   +-----------+   +-----------+
| prev: 0x..|   | prev: ... |   | prev: ... |
| txs       |-->| txs       |-->| txs       |
| state root|   | state root|   | state root|
+-----------+   +-----------+   +-----------+
     hash H98        hash H99       (tip)
```

Accounts compared:

	Externally owned	Contract
Controlled by	private key	its code
Can start a transaction	yes	no
Has a 4-byte code field	empty code	deployed bytecode
Example	your wallet	the Critter contract

Gas is the unit of computation. Common operations:

Operation	Gas (approx.)	Note
Simple add	3-10	arithmetic is cheap
Write a storage slot	20,000 (cold) / 2,900 (warm)	the expensive resource
Deploy a contract	32,000 + code size x 200	paid once
Call another contract	2,600 + execution	paid per call
Log an event	375 + 375 per topic	cheap way to store history

A transaction has a `from`, an optional `to`, a `value` in wei, a `data` field (the function selector plus arguments) and a gas limit. If execution runs out of gas, the whole transaction reverts and the sender still pays for the gas used so far.

1.3 Examples

Example 1 - Reading the chain with JSON-RPC

Every node exposes an HTTP JSON-RPC endpoint. `eth_chainId` returns the network identifier and `eth_blockNumber` the current height:

shell

```shell
curl -s -X POST https://ethereum-rpc.publicnode.com \
  -H 'content-type: application/json' \
  -d '{"jsonrpc":"2.0","id":1,"method":"eth_chainId","params":[]}'
```

Expected output (values quoted in hexadecimal, as JSON-RPC requires):

output

```
{"jsonrpc":"2.0","id":1,"result":"0x1"}
```

`0x1` means chain id 1, Ethereum mainnet. A second call to `eth_blockNumber` returns the height:

output

```
{"jsonrpc":"2.0","id":1,"result":"0x13a2b4c"}
```

Convert it with `int 0x13a2b4c` if you need a decimal height.

Example 2 - Anatomy of a transfer

Sending 0.01 ETH to a contract means value `10000000000000000` wei (0.01 x 10^18), a `data` field that starts with the function selector, and a gas limit you choose:

JSON

```
{
  "from": "0x742d35Cc6634C0532925a3b844Bc454e4438f44e",
  "to": "0x5FbDB2315678afecb367f032d93F642f64180aa3",
  "value": "0x2386f26fc10000",
  "data": "0xa9059cbb000000000000000000000000...",
  "gas": "0x5208",
  "chainId": "0x1"
}
```

Expected result: the wallet signs this object, a node includes it in a block, the contract function identified by `0xa9059cbb` (transfer) runs and the balance of the recipient increases by the value.

1.4 Common mistakes and best practices

Mistake	Why it hurts	Best practice
Treating gas as free	a storage-heavy design costs users money and can exceed the block limit	count storage writes, batch when possible
Assuming the contract can read a file or URL	EVM execution is deterministic, so any external read would fork the network	push data on chain or use an oracle (chapter 12)
Using `block.timestamp` as a clock	miners can nudge it by seconds, and it is only a timestamp	fine for cooldowns of minutes, never for randomness
Forgetting that ether has 18 decimals	`value: 1` is 1 wei, not 1 ETH	work in wei and convert only in the UI

1.5 Practice exercises

1. Explain, in two sentences, why modifying an old block is rejected by the network.
2. An externally owned account sends a transaction that calls a contract function and runs out of gas after changing one storage slot. What happens to the storage change, and who pays?
3. A contract needs the current EUR price of ETH. Why can it not fetch it with an HTTP call, and which pattern from this chapter solves the problem?

Solutions

1. Changing a block changes its hash, which invalidates the stored `prev` pointer of the next block and every block after it. Honest nodes would have to recompute all those hashes and re-validate proof of stake for the branch, so the honest chain with more work/stake wins and the edited branch is discarded.
2. The whole transaction reverts, including the storage write: state changes only become permanent when a transaction finishes successfully. The sender still pays for the gas consumed up to the revert, which compensates the nodes for the work performed.
3. Every node must reach the same result for the same inputs, but an HTTP call would return a different answer at different times and could fail, so nodes would disagree. The contract must instead read a value that was published on chain by an oracle (chapter 12).

1.6 Mini test

1. What does a block store that links it to the previous block? a) the full state of the chain b) the hash of the previous block c) the private key of the validator d) nothing; order comes from timestamps
2. True or false: contract accounts can create transactions on their own.
3. Why is a contract unable to perform an HTTPS GET?
4. One storage write costs about this much gas: fill in a number from the gas table.
5. Which unit is used for ether in transactions, and how many decimals does it have?

Answer key and explanations

1. b. Storing the previous hash makes the ledger tamper-evident, because any edit breaks the pointer to every later block.
2. False. Only externally owned accounts can originate transactions; a contract acts only when another account calls it.
3. Because execution must be deterministic across all nodes; a network request could return different data or fail per node. Oracles (chapter 12) bring off-chain data on chain.
4. About 20,000 gas for a cold slot (5,000 for a reset), which is why storage writes dominate contract cost.
5. Wei. One ETH is 10^18 wei, so ether values are always integers on chain.

Chapter 2 - Your first smart contract

2.1 Summary

- A Solidity file starts with an SPDX licence identifier and a `pragma` line that pins the compiler versions the contract accepts.
- A `contract` groups state variables (permanent storage) and functions that read or change them.

- State variables live in storage and keep their value between calls; variables declared inside a function live in memory.
- Solidity 0.8 and later revert on integer overflow and underflow by default instead of wrapping silently.
- The `constructor` runs exactly once, at deployment time, and typically receives the creator address and initial settings.
- Value types include `bool`, integers of eight sizes, `address`, `bytes32` and enums; reference types are arrays, structs, mappings and strings.
- A `constant` or `immutable` variable is baked into the code and costs no gas slot when written once at deployment.
- Deploying means sending a creation transaction whose data is the compiled bytecode plus encoded constructor arguments.

2.2 Concepts

File skeleton:

output

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.24;

contract Name {
    // 1. state variables
    // 2. events
    // 3. modifiers
    // 4. constructor
    // 5. external functions
    // 6. public / internal functions
}
```

Integer types and their use:

Type	Bits	Use for
`uint256`	256	counters, balances, ids (the default)
`uint128`, `uint64`, `uint32`	smaller	packed storage structs
`int256`	256	signed deltas, temperature, PnL
`uint8`	8	percentages, small levels (max 255)
`bool`	1 slot when packed	flags
`address`	160 bits	accounts and contracts
`bytes32`	256 bits	hashes, short keys
`enum`	index	states of a state machine

Checked arithmetic in Solidity 0.8 means `255 + 1` on a `uint8` reverts the whole call. `unchecked { }` disables the check and is only for loops and proven safe math, because it saves gas.

Memory versus storage: function arguments that are structs or arrays must be marked `memory` (temporary copy) or `calldata` (read-only view of call data); state variables always live in storage.

2.3 Examples

Example 1 - A counter contract

Solidity

```solidity
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.24;

/// @title A minimal counter
/// @notice Anyone can increment; only the counter owner can reset it
contract Counter {
    uint256 public count;        // state variable, starts at 0
    address public owner;         // set in the constructor
    uint256 public constant MAX = 1000; // no storage slot needed

    event CountChanged(address indexed by, uint256 newCount);

    constructor() {
        owner = msg.sender;      // the deployer becomes the owner
    }

    function increment() external {
        require(count < MAX, "Counter: limit reached");
        count += 1;              // checked arithmetic, reverts on overflow
        emit CountChanged(msg.sender, count);
    }

    function reset() external {
        require(msg.sender == owner, "Counter: not owner");
        count = 0;
        emit CountChanged(msg.sender, count);
    }

    function getCount() external view returns (uint256) {
        return count;            // a read-only call, costs no gas to the caller
    }
}
```

Expected behaviour when deployed to Remix and called from account A:

output

```
constructor runs         -> owner = 0x5FbDB...aaC3
increment() from A       -> count = 1, event CountChanged(0x5FbDB..., 1)
increment() from B       -> count = 2, event CountChanged(0xAb85..., 2)
getCount()               -> 2
reset() from B           -> reverted: "Counter: not owner"
reset() from A           -> count = 0
increment() 1001 times   -> reverted: "Counter: limit reached"
```

Example 2 - Overflow protection

Solidity

```solidity
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.24;

contract Wrap {
    uint8 public small = 250;

    function add(uint8 a, uint8 b) external pure returns (uint8) {
        return a + b;            // reverts if the true value exceeds 255
    }

    function addUnchecked(uint8 a, uint8 b) external pure returns (uint8) {
        unchecked { return a + b; } // wraps: 250 + 10 returns 4
    }
}
```

Expected behaviour:

output

```
add(250, 10)              -> reverted (arithmetic overflow)
addUnchecked(250, 10)     -> 4
add(200, 55)              -> 255
```

2.4 Common mistakes and best practices

Mistake	Why it hurts	Best practice
Omitting the SPDX line	modern solc emits a warning and some tools refuse to compile	always put `// SPDX-License-Identifier: MIT` first
Using `uint` where `uint256` fits	smaller types only help when several pack into one slot	default to `uint256`, shrink inside structs
Leaving old-style constructors with the contract name	they are deprecated and confusing	use `constructor()`
Counting on `require` messages as user errors	messages cost bytecode size and are rarely surfaced	short ASCII messages, meaningful names

2.5 Practice exercises

1. Write a `Box` contract that stores a `uint256` value, lets anyone set a new value only when it is strictly greater than the current one, and emits an event per change.
2. Add a function `isOwner()` that returns whether the caller is the deployer, without writing a new storage variable.
3. In the `Wrap` contract, what does `addUnchecked(0, 0)` return and why?

Solutions

1.

Solidity

```solidity
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.24;

contract Box {
    uint256 public value;
    address public immutable deployer;
    event ValueChanged(uint256 newValue, address changedBy);

    constructor() {
        deployer = msg.sender;
    }

    function setValue(uint256 newValue) external {
        require(newValue > value, "Box: must increase");
        value = newValue;
        emit ValueChanged(newValue, msg.sender);
    }

    function isOwner() external view returns (bool) {
        return msg.sender == deployer;
    }
}
```

Expected behaviour: `setValue(5)` sets `value = 5`; `setValue(3)` reverts with `"Box: must increase"`; `isOwner()` returns `true` for the deployer and `false` for any other account.

2. Comparing `msg.sender` with a stored/deployed address needs no extra variable only if the deployer address is already stored, so use `immutable deployer` as in the solution above. Otherwise store `owner` once in the constructor.
3. `0`. Unchecked arithmetic wraps modulo 256, and `0 + 0 = 0`, so nothing changes.

2.6 Mini test

1. Which two lines are mandatory (or warning-free) at the top of every modern Solidity file?
2. True or false: `uint8` arithmetic wraps silently in Solidity 0.8.
3. Where does a local `uint256 x` declared inside a function live?
4. What keyword makes a variable that is written once in the constructor and never again?
5. When exactly does the constructor run?

Answer key and explanations

1. The `// SPDX-License-Identifier: MIT` comment and the `pragma solidity ^0.8.24;` version line.
2. False. Since 0.8 overflow reverts the transaction; wrapping requires an explicit `unchecked` block.
3. In memory: it is a temporary value that disappears when the call ends. Only state variables persist in storage.
4. `immutable`. It is stored in the bytecode area rather than a storage slot, so reads are cheap.
5. Exactly once, during deployment, in the same transaction that creates the contract code.

Chapter 3 - Data structures: structs and arrays

3.1 Summary

- A `struct` is a custom type that groups several fields under one name, which is how you model domain objects on chain.
- Arrays can be fixed length (`uint[5]`) or dynamic (`uint[]`) and are reference types, so passing them around needs a data location keyword.
- `array.push(x)` appends to a dynamic storage array; `array.length` is the element count, not its byte size.
- Storage arrays are writable and expensive; memory arrays are scratch copies used inside a call.
- A `mapping` maps keys to values with O(1) lookup and cannot be iterated: you must track the keys yourself if you need a list.
- Mapping deletion is partial: `delete m[key]` resets the value but leaves the key in place.
- Nested structures (array of structs, mapping to a struct holding an array) are the standard way to model a collection of items.
- Because every storage write costs gas, modelling data decides most of the contract's cost.

3.2 Concepts

Structs:

output

```
struct Critter {
    string  name;
    uint32  level;
    uint32  cooldownEndsAt;
    uint8   wins;
    bool    ready;
}
```

Each field occupies a storage slot, but Solidity packs fields smaller than 32 bytes into a shared slot: `level`, `cooldownEndsAt`, `wins` and `ready` above fit into one slot if declared together.

Arrays compared:

Form	Example	Notes
Dynamic storage	`Critter[] critters;`	`push`, `pop`, `length`, persistent
Fixed storage	`address[4] winners;`	all elements always exist (zero values)
Memory	`uint[] memory tmp = new uint[](3);`	temporary, allocated per call

Form	Example	Notes
Calldata	`uint[] calldata data`	read-only view of arguments, gas-free copy
Bytes	`bytes` / `string`	dynamic, packed, expensive to compute length

Mappings:

Solidity

```
mapping(address => uint256) public balances;      // address -> balance
mapping(uint256 => Critter) public critters;      // id -> struct
mapping(address => mapping(address => bool)) approved; // nested allowance
```

Never iterate a mapping. If you need "all ids of this owner", store an array of ids per owner and clean it up on transfer, as in chapter 7.

Data location rules in short: state variables and returns to storage need no keyword, function parameters that are reference types must say `memory` or `calldata`, and assigning a storage struct to memory copies it (changes do not write back).

3.3 Examples

Example 1 - A struct-based registry

Solidity

```solidity
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.24;

/// @title A minimal registry of critters with lookup by id
contract CritterRegistry {
    struct Critter {
        string name;
        uint32 level;
        uint32 cooldownEndsAt;
        uint8 wins;
        bool ready;
    }

    Critter[] private critters;                     // id = index
    mapping(address => uint256[]) private ownedIds;

    event CritterCreated(uint256 indexed id, string name, address indexed owner);

    function createCritter(string calldata name) external returns (uint256 id) {
        id = critters.length;
        critters.push(Critter({
            name: name,
            level: 1,
            cooldownEndsAt: 0,
            wins: 0,
            ready: true
        }));
        ownedIds[msg.sender].push(id);
        emit CritterCreated(id, name, msg.sender);
    }

    function getCritter(uint256 id) external view returns (Critter memory) {
        require(id < critters.length, "Registry: no such critter");
        return critters[id];            // returns a memory copy
    }

    function count() external view returns (uint256) {
        return critters.length;
    }

    function crittersOf(address owner) external view returns (uint256[] memory) {
        return ownedIds[owner];         // memory copy again
    }

    function levelUp(uint256 id) external {
        require(id < critters.length, "Registry: no such critter");
        Critter storage c = critters[id];       // storage reference, not a copy
        require(c.ready, "Registry: cooling down");
        c.level += 1;
    }
}
```

Expected output (calls from account A, then B):

output

```
createCritter("Fluffy") from A -> id 0, event CritterCreated(0, "Fluffy", 0x5FbDB...)
createCritter("Sparky") from A -> id 1
createCritter("Pebble") from B -> id 2
getCritter(1)                  -> ("Sparky", 2, 0, 0, true)  after levelUp(1)
crittersOf(A)                  -> [0, 1]
crittersOf(B)                  -> [2]
levelUp(5)                     -> reverted: "Registry: no such critter"
```

Example 2 - Mappings instead of arrays

Solidity

```solidity
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.24;

contract Vault {
    mapping(address => uint256) private balances;
    address[] private participants;              // key list, needed to iterate

    function deposit() external payable {
        require(msg.value > 0, "Vault: send ether");
        if (balances[msg.sender] == 0) {
            participants.push(msg.sender);
        }
        balances[msg.sender] += msg.value;
    }

    function balanceOf(address who) external view returns (uint256) {
        return balances[who];
    }

    function totalParticipants() external view returns (uint256) {
        return participants.length;
    }

    function reset(address who) external {
        delete balances[who];    // value becomes 0; the key stays
    }
}
```

Expected output:

output

```
deposit() with 1 ether from A  -> balances[A] = 1e18 wei
deposit() with 2 ether from A  -> balances[A] = 3e18 wei, participants = [A]
deposit() with 5 ether from B  -> balances[B] = 5e18, participants = [A, B]
reset(A)                       -> balanceOf(A) = 0, totalParticipants() still 2
```

Note that `deposit` is `payable`, so it can receive ether; the accounting pattern is used again in chapter 8.

3.4 Common mistakes and best practices

Mistake	Why it hurts	Best practice
Copying a storage struct into memory to edit it	edits are lost, because memory is a copy	use `Struct storage s = arr[i];` to edit in place

Mistake	Why it hurts	Best practice
Trying to iterate a mapping	mappings expose no length or keys	keep a key array alongside the mapping
Storing a `string` when a hash suffices	strings are dynamic and expensive	store `bytes32` hashes when you only need comparison
Declaring fields that never pack together	each 32-byte slot costs 20k gas to write	group small fields (uint32, uint8, bool) side by side

3.5 Practice exercises

1. Add `function removeCritter(uint256 id)` to `CritterRegistry` that marks the struct's `ready` flag false and records the caller as a new `address` field, and explain why a full delete of the array element is avoided.
2. Write `function latestId() external view returns (uint256)` that returns the newest id or reverts when the registry is empty.
3. In `Vault`, a user calls `deposit()` with 0 ether. What happens and why is the check there?

Solutions

1.

Solidity

```
    function removeCritter(uint256 id) external {
        require(id < critters.length, "Registry: no such critter");
        Critter storage c = critters[id];
        c.ready = false;
        c.wins = 0;
    }
```

Deleting the array element would shift every later element and change the ids already referenced elsewhere, so instead you flag it inactive and filter it in the UI. A full `delete critters[id]` would also destroy the record other accounts reference. 2.

Solidity

```
    function latestId() external view returns (uint256) {
        require(critters.length > 0, "Registry: empty");
        return critters.length - 1;
    }
```

Expected output: `0` after one creation, `reverted: "Registry: empty"` on a fresh deployment.

3. `deposit()` reverts with `"Vault: send ether"`, because `msg.value` is 0 and the check protects the accounting: a zero-value call would otherwise add the address to `participants` with a zero balance.

3.6 Mini test

1. What does `array.length` return for a dynamic array and what is its maximum?

2. Which keyword gives you a writable reference into storage instead of a copy?
3. True or false: a mapping can tell you how many keys it contains.
4. Which two data locations are valid for a function parameter that is an array?
5. Why pack `uint32` fields side by side in a struct?

Answer key and explanations

1. The number of elements (for `bytes`, the byte length); it can reach 2^256-1, so it is effectively unlimited but each element costs storage.
2. `storage`, as in `Critter storage c = critters[i];`.
3. False. Mappings expose no keys and no length; you must maintain a separate key array to iterate.
4. `memory` (a writable copy) and `calldata` (a read-only view of the call arguments, cheaper for external functions).
5. Because fields smaller than 32 bytes sharing a slot are written with one 20k-gas slot instead of one per field.

Chapter 4 - Functions, visibility and events

4.1 Summary

- A function declaration names its visibility: `external` (from outside only), `public` (inside and outside), `internal` (this contract and children) or `private` (this contract only).
- `view` functions read state without changing it; `pure` functions do not even read state; both are free to call from off chain.
- `payable` functions can receive ether with the call; without it any value sent is rejected.
- Events are cheap storage: they write data to transaction logs that off-chain services and front-ends subscribe to.
- An `event` marks parameters `indexed` so clients can filter logs by up to three values.
- Function parameters may be `memory` (writable copy), `calldata` (read-only, cheaper) or storage references.
- Return values can be named, which documents intent and enables an early `return` without repeating variables.
- Custom errors (`error Unauthorized(address caller);` + `revert Unauthorized(msg.sender);`) cost less gas than long `require` strings and carry structured data.
- A `modifier` runs code before and after a function, the classic use being access control such as `onlyOwner`.

4.2 Concepts

Visibility matrix:

Keyword	Inside	Derived contracts	Outside
`private`	yes	no	no
`internal`	yes	yes	no
`external`	no (use `this.f()`)	yes	yes
`public`	yes	yes	yes

State mutability:

Kind	Reads state	Writes state	Costs gas off-chain call
`pure`	no	no	no
`view`	yes	no	no
non-payable	yes	yes	yes
`payable`	yes	yes	yes, can carry value

Logs versus storage: a storage write costs about 20,000 gas and stays in the state; an event costs roughly 375 gas plus 375 per indexed topic, lives in the receipt logs, and is readable by any client for the cost of an RPC query. Use events for history, storage for data the contract itself must read back.

Modifiers and the function flow: the modifier body runs up to the _ placeholder, then the function executes:

output

```
caller -> onlyOwner check -> _ -> function body -> modifier tail (if any)
```

4.3 Examples

Example 1 - Events and visibility in a registry

Solidity

```solidity
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.24;

/// @title Shows visibility, view functions, events and a modifier
contract Registry {
    address public owner;
    uint256 public itemCount;
    mapping(uint256 => string) private items;

    event ItemAdded(uint256 indexed id, string label, address indexed author);
    event OwnerChanged(address indexed previousOwner, address indexed newOwner);

    error NotOwner(address caller);

    modifier onlyOwner() {
        if (msg.sender != owner) revert NotOwner(msg.sender);
        _;
    }

    constructor() {
        owner = msg.sender;
        emit OwnerChanged(address(0), msg.sender);
    }

    function addItem(string calldata label) external {
        uint256 id = itemCount;
        items[id] = label;
        itemCount = id + 1;
        emit ItemAdded(id, label, msg.sender);
    }

    function getItem(uint256 id) external view returns (string memory) {
        return items[id];               // view: no gas when called off-chain
    }

    function isOwner(address who) external view returns (bool) {
        return who == owner;            // view: it reads state
    }

    function transferOwnership(address newOwner) external onlyOwner {
        emit OwnerChanged(owner, newOwner);
        owner = newOwner;
    }
}
```

A `pure` function cannot touch state at all, which is why `sum` below is pure while `isOwner` above is only `view`:

Solidity

```solidity
function sum(uint256 a, uint256 b) external pure returns (uint256) {
    return a + b;                       // pure: only its arguments are used
}
```

Expected behaviour (Remix, deployed by account A):

output

```
addItem("first")  from A   -> ItemAdded(0, "first", 0x5FbDB...), itemCount = 1
getItem(0)                 -> "first"
sum(2, 3)                  -> 5
addItem("x")    from B     -> succeeds: addItem is public to everyone
transferOwnership(B) from B-> reverted with NotOwner(0xAb85...)
transferOwnership(B) from A-> OwnerChanged(0x5FbDB..., 0xAb85...), owner = B
addItem("x")    from B     -> now allowed because B owns the registry
```

Example 2 - Listening to events from the front-end

JavaScript

```javascript
// web3.js v4 style; the ABI comes from the compiler's output
const contract = new web3.eth.Contract(abi, deployedAddress);

contract.events.ItemAdded({ fromBlock: 0 })
  .on('data', (event) => {
    console.log(`item ${event.returnValues.id}: ${event.returnValues.label}`);
  })
  .on('error', console.error);
```

Expected output in the browser console after the three calls above:

output

```
item 0: first
```

Events keep coming as new blocks arrive, so the log grows every time anyone adds an item.

4.4 Common mistakes and best practices

Mistake	Why it hurts	Best practice
Marking a state-reading function `pure`	it does not compile	`view` when it reads state, `pure` when it does not
Long `require("...")` messages	they inflate bytecode and are rarely read	short messages, or custom errors with parameters
Forgetting `indexed` on filter fields	clients must scan every log to find one event	index the fields you will query by (max 3)
Expecting events to be readable by the contract	logs are not state; the contract cannot read them back	keep authoritative data in storage, mirror it to events
Using `public` for internal helpers	exposes them on-chain and blocks future changes	`internal`/`private` until needed

4.5 Practice exercises

1. Add `function removeItem(uint256 id) external onlyOwner` to the registry that clears an item and emits `ItemRemoved(uint256 indexed id)`; declare the event too.
2. Why can't `sum(2, 3)` be called with ether attached, and what happens if you try?

3. Change `addItem` so only the owner may add items, using the existing modifier. What happens to account B's ability to add items?

Solutions

1.

Solidity

```
    event ItemRemoved(uint256 indexed id);

    function removeItem(uint256 id) external onlyOwner {
        delete items[id];
        emit ItemRemoved(id);
    }
```

Expected output: called by the owner, `ItemRemoved(0)` is logged and a later `getItem(0)` returns an empty string; called by anyone else it reverts.

2. `sum` is a non-payable `pure` function, and the compiler inserts a check that rejects calls carrying value. Sending ether makes the whole transaction revert, and the caller only loses gas.
3. Add `onlyOwner` to `addItem` (`function addItem(...) external onlyOwner`). Account B's calls then revert with `NotOwner` until an owner grants B ownership through `transferOwnership`, because the modifier checks `msg.sender` on every call.

4.6 Mini test

1. Which visibility allows calls from derived contracts but not from outside?
2. True or false: a `view` function that writes nothing can still cost gas when called from another contract.
3. How many event parameters can be `indexed` at most?
4. What is the difference between `memory` and `calldata` for a function parameter?
5. Which built-in variable tells you the address that called the current function?

Answer key and explanations

1. `internal`: available inside the contract and in contracts that inherit it, invisible to external callers.
2. True. Off-chain calls of view functions are free, but an on-chain contract still pays for the execution.
3. Three: `indexed` parameters become topics, and a log entry can carry at most three.
4. `calldata` is a read-only view of the call arguments and is cheaper; `memory` is a writable temporary copy.
5. `msg.sender`. (For the original transaction sender, unaffected by contract-to-contract calls, you would need `tx.origin` - which you should almost never use.)

Chapter 5 - Ownership, mappings and security basics

5.1 Summary

- Every call knows `msg.sender`; a contract that stores its creator in a state variable can restrict functions to that owner.
- The ownable pattern is the smallest access control: a single owner who can transfer the role.
- `require(condition, "reason")` reverts the transaction when the condition is false, undoing every state change in the call.
- Reentrancy is the attack where a malicious contract calls back into your contract before you finished updating its balances.
- The checks-effects-interactions order (validate, update storage, then call outside) defeats reentrancy in most contracts.
- A reentrancy guard (a mutex flag) adds a second layer for functions that must call untrusted addresses.
- Pull payments (let users withdraw instead of pushing ether to them) are safer than loops that send to many addresses.
- `transfer`ing ether with `.call{value: amount}("")` forwards all remaining gas, so it must be paired with effects-first ordering.
- Mapping-based balances (chapter 3) plus a withdrawal function is the standard escrow pattern used by games and sales.

5.2 Concepts

Why reentrancy works: a call to another contract hands over control. If your contract sends ether before updating the ledger, the receiver can call withdraw again while its balance still reads non-zero:

output

```
VULNERABLE
1. user withdraw()        -> balance[user] = 1 ether
2. contract sends 1 ether -> receiver's fallback runs
3. fallback calls withdraw() again, balance still 1 ether
4. step 2 repeats until the contract is drained

SAFE (checks-effects-interactions)
1. check: require(balance[user] >= amount)
2. effect: balance[user] = 0
3. interaction: send the ether (even a reentrant call now sees 0)
```

The guard pattern:

Solidity

```
bool private locked;

modifier nonReentrant() {
    require(!locked, "Reentrant call");
    locked = true;
    _;
    locked = false;
}
```

Note that the simple version above is outdated; OpenZeppelin's `ReentrancyGuard` is the production choice, but it illustrates the idea.

5.3 Examples

Example 1 - Ownable plus a safe vault

Solidity

```solidity
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.24;

/// @title Owner-restricted contract with a pull-payment vault
contract OwnedVault {
    address public owner;
    mapping(address => uint256) public balances;
    uint256 public totalDeposited;

    event Deposited(address indexed who, uint256 amount);
    event Withdrawn(address indexed who, uint256 amount);
    event OwnerTransferred(address indexed from, address indexed to);

    error NotOwner(address caller);
    error NothingToWithdraw();

    modifier onlyOwner() {
        if (msg.sender != owner) revert NotOwner(msg.sender);
        _;
    }

    constructor() {
        owner = msg.sender;
    }

    function deposit() external payable {
        require(msg.value > 0, "Vault: zero deposit");
        balances[msg.sender] += msg.value;          // effect first
        totalDeposited += msg.value;
        emit Deposited(msg.sender, msg.value);
    }

    function withdraw() external {
        uint256 amount = balances[msg.sender];      // check
        if (amount == 0) revert NothingToWithdraw();
        balances[msg.sender] = 0;                   // effect before interaction
        (bool sent, ) = msg.sender.call{value: amount}("");
        require(sent, "Vault: transfer failed");
        emit Withdrawn(msg.sender, amount);
    }

    function sweep(address to) external onlyOwner {
        require(to != address(0), "Vault: zero address");
        (bool sent, ) = to.call{value: address(this).balance}("");
        require(sent, "Vault: sweep failed");
    }

    function transferOwnership(address newOwner) external onlyOwner {
        require(newOwner != address(0), "Vault: zero owner");
        emit OwnerTransferred(owner, newOwner);
        owner = newOwner;
    }

    receive() external payable {
        balances[msg.sender] += msg.value;          // plain ether sends are deposits
        totalDeposited += msg.value;
```

Solidity (continued)

```solidity
        emit Deposited(msg.sender, msg.value);
    }
}
```

Expected output (account A deploys, B interacts):

output

```
deposit() with 1 ether from B -> balances[B] = 1e18, totalDeposited = 1e18
deposit() with 0 wei from B   -> reverted: "Vault: zero deposit"
withdraw() from B             -> 1 ether sent, balances[B] = 0, Withdrawn(B, 1e18)
withdraw() from B again       -> reverted: NothingToWithdraw()
sweep(C) from B               -> reverted: NotOwner(B)
sweep(C) from A               -> contract balance moved to C
sending 2 ether to the address (no data)
                               -> receive() runs, balances[sender] = 2e18
```

Example 2 - The vulnerable version, for contrast

Solidity

```solidity
// DO NOT USE - vulnerable to reentrancy
contract BadBank {
    mapping(address => uint256) public balances;

    function deposit() external payable {
        balances[msg.sender] += msg.value;
    }

    function withdraw() external {
        uint256 amount = balances[msg.sender];
        (bool sent, ) = msg.sender.call{value: amount}("");  // interaction FIRST
        require(sent, "send failed");
        balances[msg.sender] = 0;                            // effect AFTER
    }
}
```

Expected result: an attacker contract whose `receive()` calls `withdraw()` again drains the bank, because the balance is still non-zero when the external call runs. This is exactly the pattern of the 2016 DAO incident, and the reason effects must come before interactions.

5.4 Common mistakes and best practices

Mistake	Why it hurts	Best practice
Sending ether before updating balances	reentrancy can drain the contract	checks-effects-interactions, always
Looping `.call` sends to many users	one failing address blocks the payout for everyone	pull payments: credit balances, users withdraw
Leaving `owner` as a deployer EOA with no plan	the contract is un-upgradeable and unkillable if the key is lost	transfer ownership to a multisig or governance
Using `tx.origin` for authorization	phishing contracts can relay the caller's authority	use `msg.sender`
Trusting `transfer`/`send` gas stipends	2300 gas is not enough for a multisig receiver	`.call{value: ...}` with reentrancy guard

5.5 Practice exercises

1. In `OwnedVault`, why does the reentrant attacker now fail? Answer in one sentence.
2. Add `event OwnerTransferred` misuse check: what happens when `transferOwnership(address(0))` is called?
3. Rewrite `withdraw` so it works with `nonReentrant` style guard, showing just the modifier and the function signature.

Solutions

1. Because `withdraw` sets `balances[msg.sender] = 0` before the `.call`, the attacker's `receive()` runs when the recorded balance is already zero, so the recursive call reverts on `NothingToWithdraw` or sends zero.
2. The `require(newOwner != address(0), ...)` rejects the zero address and reverts before the state change, so ownership can never be burned to an unusable address.
3.

Solidity

```solidity
    bool private locked;

    modifier nonReentrant() {
        require(!locked, "Vault: reentrant");
        locked = true;
        _;
        locked = false;
    }

    function withdraw() external nonReentrant {
        // body as before: check, effect, interaction
    }
```

Expected behaviour: the second nested call reverts immediately with `"Vault: reentrant"`, so only one withdrawal per transaction succeeds.

5.6 Mini test

1. Which order of operations stops reentrancy?
2. True or false: `require` reverts all state changes made earlier in the same call.
3. What is the standard way to pay many users so a single failure cannot block the others?
4. Which variable identifies the immediate caller of the current function?
5. Why is `tx.origin` dangerous for access control?

Answer key and explanations

1. Checks (validate inputs), effects (update storage), interactions (call other contracts) — effects before interactions.
2. True. A revert rolls back every write performed during that call and its sub-calls.
3. Pull payments: record what each user may claim, then let each user withdraw.
4. `msg.sender`.

5. It holds the original transaction's sender, so a malicious contract in between can ask the victim to sign a transaction that the contract then exploits (classic phishing pattern).

Chapter 6 - Inheritance, interfaces and imports

6.1 Summary

- Solidity contracts inherit with `is`: `contract Dog is Animal`, and a single contract can inherit several bases (`contract X is A, B, C`).
- The base contract must be fully implemented (or declared `abstract`) before a derived contract can be deployed.
- A base function must be `virtual` to be overridden; the override must repeat `override` (and list the base contracts when multiple bases define it).
- State variables are inherited too, so derived contracts read and write the base storage slots.
- Constructors of base contracts run in dependency order, most-base first, before the derived constructor.
- An `interface` declares functions without bodies and is the standard way to talk to contracts you do not control.
- `abstract` contracts contain unimplemented functions and cannot be deployed, only inherited from.
- Imports pull in other files: local paths (`"./Ownable.sol"`) or package paths (`"@openzeppelin/contracts/..."`).
- Multiple inheritance uses C3 linearisation; when two bases define the same function, the override must name them: `override(BaseA, BaseB)`.

6.2 Concepts

Inheritance and override rules:

output

```
contract Base {
    function feed() public virtual returns (string memory);
}

contract Middle is Base {
    function feed() public virtual override returns (string memory) { ... }
}

contract Derived is Middle {
    function feed() public override returns (string memory) { ... }
}
```

Interfaces: no state, no function bodies, external functions only, everything implicitly virtual. They exist so your contract can call a known ABI at a known address:

Solidity

```solidity
interface IUniswapV2Factory {
    function getPair(address tokenA, address tokenB) external view returns (address pair);
}
```

The contract then only needs the address: `IUniswapV2Factory(addr).getPair(a, b)`.

Import forms:

Form	Meaning
`import "./File.sol";`	all symbols from a local file
`import {Ownable, Ownable2Step} from "@openzeppelin/contracts/access/Ownable.sol";`	named symbols from a package
`import "@openzeppelin/contracts/token/ERC721/ERC721.sol" as ERC;`	namespaced import

6.3 Examples

Example 1 - Overriding and constructors

Solidity

```solidity
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.24;

abstract contract Animal {
    string public name;
    uint256 public legs;

    constructor(string memory _name) {
        name = _name;
    }

    function sound() public pure virtual returns (string memory);

    function describe() public view returns (string memory) {
        return sound();
    }
}

contract Dog is Animal {
    constructor(string memory _name) Animal(_name) {   // base constructor first
        legs = 4;
    }

    function sound() public pure override returns (string memory) {
        return "woof";
    }
}

contract Puppy is Dog {
    constructor() Dog("Rex") {}                        // most-base first

    function sound() public pure override returns (string memory) {
        return "yip";
    }
}
```

Expected behaviour (Remix, deploy `Puppy`):

output

```
name()                    -> "Rex"     (from Dog's base constructor argument)
legs()                    -> 4         (set by Dog's constructor)
sound()                   -> "yip"     (Puppy overrides Dog)
describe()                -> "yip"     (calls the most-derived override)
deploying Animal directly -> rejected: abstract contracts cannot be deployed
```

Example 2 - Interfaces for calling other contracts

Solidity

```solidity
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.24;

interface IERC20 {
    function transfer(address to, uint256 amount) external returns (bool);
    function balanceOf(address who) external view returns (uint256);
}

/// @notice Lets anyone swap an ERC20 token 1:1 for ether at a fixed 1 gwei price
contract SimpleSeller {
    IERC20 public immutable token;

    constructor(address tokenAddress) {
        token = IERC20(tokenAddress);
    }

    function buy(uint256 amount) external payable {
        require(msg.value == amount * 1 gwei, "Seller: wrong price");
        require(token.balanceOf(address(this)) >= amount, "Seller: short stock");
        require(token.transfer(msg.sender, amount), "Seller: transfer failed");
    }
}
```

Expected behaviour (a token deployed first, then the seller with its address):

output

```
approve(seller, 100e18) on the token
buy(100e18) with 0.1 ether attached -> token moves to the buyer, seller holds the ether
buy(100e18) with 0.09 ether        -> reverted: "Seller: wrong price"
buy(1e24)                         -> reverted: "Seller: short stock"
```

Note the return value check: an ERC20 that silently returns `false` (or nothing) must not be treated as a success, which is why `transfer` is wrapped in `require`.

6.4 Common mistakes and best practices

Mistake	Why it hurts	Best practice
Forgetting `virtual` on the base function	overriding does not compile	mark bases `virtual`, overrides `override`
Deploying a contract that leaves a function without a body	transactions revert and the address stays broken	make it `abstract` until every function is implemented
Passing constructor arguments in the wrong order	silent wrong configuration	always test the constructor path
Calling an unknown address without an interface	easy typos, no compile-time check	declare an interface and cast the address
Ignoring the return value of external calls	some tokens return `false` instead of reverting	check the boolean and revert yourself

6.5 Practice exercises

1. Make `Puppy` inherit a second base contract `Toy` that adds `function material() public pure virtual returns (string memory)` and override it in `Puppy`.
2. Why must `Animal` be declared `abstract` in Example 1?
3. In `SimpleSeller`, what protects the buyer if `transfer` silently returns `false`?

Solutions

1.

Solidity

```
contract Toy {
    function material() public pure virtual returns (string memory) {
        return "rope";
    }
}

contract Puppy is Dog, Toy {
    constructor() Dog("Rex") {}

    function sound() public pure override returns (string memory) {
        return "yip";
    }
}
```

Expected behaviour: `sound()` returns `"yip"`, `material()` returns `"rope"` inherited from `Toy`, and the deployment still sets `name = "Rex"`.

2. Because `sound()` has no implementation; a concrete contract must implement every function it inherits, so `Animal` stays abstract and only derived contracts are deployable.
3. `require(token.transfer(...), "Seller: transfer failed")`: if the token returns `false` or throws, the whole purchase reverts and the buyer keeps both the ether and the tokens.

6.6 Mini test

1. Which two keywords pair an overridable base function with its override?
2. True or false: an interface can contain state variables.
3. In what order do base constructors run relative to the derived one?
4. How do you call a function on a known contract whose code you do not have?
5. What happens when two bases define the same function and the derived contract overrides it?

Answer key and explanations

1. `virtual` on the base function and `override` on the derived one.
2. False. Interfaces only declare external function signatures; no state, no bodies.
3. Most-base first, ending with the derived contract's own constructor.
4. Declare an `interface` with the functions you need and cast the address to it.
5. The override must be written `override(BaseA, BaseB)` so Solidity can linearise the inheritance graph.

Chapter 7 - Tokens and the ERC721 standard

7.1 Summary

- A token is just a contract that keeps a mapping of owners and offers transfer functions; both ERC20 tokens and NFTs follow this model.
- ERC20 tokens are fungible: every unit is interchangeable, like coins in an account.
- ERC721 tokens are non-fungible: each token has a unique id and can carry different metadata, like a collectible.
- ERC721 adds approvals (`approve`, `setApprovalForAll`) so marketplaces can move tokens on the owner's behalf.
- Metadata lives off chain, usually as JSON at the URL returned by `tokenURI(id)`, because storing files on chain is expensive.
- OpenZeppelin's `ERC721` contract implements the standard correctly; you extend it and add your own minting and rules.
- Minting creates a new token id and assigns it to an address through an internal `_safeMint` or `_mint` call.
- `ownerOf(id)` and `balanceOf(address)` read ownership; `transferFrom` moves a token and must respect the approval rules.
- Royalties (EIP-2981) and the ERC721A gas-optimised variant are the usual next steps after the standard.
- Important: every ERC20/ERC721 detail lives in EIP documents (EIP-20, EIP-721), and compatible contracts follow them exactly.

7.2 Concepts

Fungible versus non-fungible:

	ERC20	ERC721
Unit	identical	unique id
Balance of address	`uint256` amount	count of owned ids
Divisible	usually 18 decimals	not divisible
Transfer	`transfer(to, amount)`	`transferFrom(from, to, id)`
Metadata	token contract level	per token id
Typical use	currencies, points, stakes	art, game items, tickets, names

The core ERC721 functions every implementation must provide:

output

```
balanceOf(address)          -> number of tokens owned
ownerOf(uint256 tokenId)    -> current owner
approve(address, uint256)   -> allow one address to move one token
setApprovalForAll(address,bool) -> allow an operator for all tokens
getApproved(uint256)        -> who may move this token
isApprovedForAll(address,address) -> operator permission
transferFrom(from,to,id)    -> move a token (checks approval)
safeTransferFrom(from,to,id)-> same, plus recipient contract checks
```

`tokenURI(id)` returns a URL (or a `data:application/json;base64,...` string) with `name`, `description`, `image` and optional attributes - the JSON shape marketplaces read.

7.3 Examples

Example 1 - Minting NFTs with OpenZeppelin

Solidity

```solidity
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.24;

import "@openzeppelin/contracts/token/ERC721/ERC721.sol";
import "@openzeppelin/contracts/access/Ownable.sol";

/// @title Critter collectible: one token per minted id, 0.01 ETH each
contract CritterNFT is ERC721, Ownable {
    uint256 public constant PRICE = 0.01 ether;
    uint256 public nextId;
    string private baseURI;
    uint256 public constant MAX_SUPPLY = 10000;

    event CritterMinted(uint256 indexed id, address indexed to, string name);

    error SoldOut();
    error WrongPayment();
    error NotTokenOwner();

    constructor(string memory baseURI_)
        ERC721("Crypto Critters", "CRIT")
        Ownable(msg.sender)
    {
        baseURI = baseURI_;           // e.g. "https://example.com/meta/"
    }

    function mint(string calldata critterName) external payable returns (uint256 id) {
        if (nextId >= MAX_SUPPLY) revert SoldOut();
        if (msg.value != PRICE) revert WrongPayment();

        id = nextId;
        nextId = id + 1;
        _safeMint(msg.sender, id);    // standard bookkeeping + ownership
        emit CritterMinted(id, msg.sender, critterName);
    }

    function ownerMint(address to) external onlyOwner returns (uint256 id) {
        if (nextId >= MAX_SUPPLY) revert SoldOut();
        id = nextId;
        nextId = id + 1;
        _safeMint(to, id);
        emit CritterMinted(id, to, "team mint");
    }

    function _baseURI() internal view override returns (string memory) {
        return baseURI;
    }

    function setBaseURI(string calldata newBase) external onlyOwner {
        baseURI = newBase;
    }
}
```

Expected behaviour (deployed by the team account T; user A calls `mint`):

output

```
mint("Fluffy") from A with 0.01 ETH -> token 0 owned by A
tokenURI(0)                         -> "https://example.com/meta/0"
ownerOf(0)                          -> A's address
balanceOf(A)                        -> 1
mint("Sparky") from B with 0.005 ETH-> reverted: WrongPayment()
mint("Sparky") from B with 0.01 ETH-> token 1 owned by B
transferFrom(A, B, 0) from A        -> ownerOf(0) = B, balanceOf(A) = 0
transferFrom(A, B, 0) from C        -> reverted by ERC721: caller is not approved
approve(C, 0) from A, then transferFrom(A, C, 0) from C -> succeeds
mint after 10000 tokens             -> reverted: SoldOut()
```

The `mint` function accepts a `critterName` but only logs it in this example: storing strings per token costs gas, so real projects keep names in their metadata JSON or a server database.

Example 2 - Reading token data from a front-end

JavaScript

```javascript
const contract = new web3.eth.Contract(nftAbi, nftAddress);

const total = await contract.methods.nextId().call();
const owner = await contract.methods.ownerOf(0).call();
const uri = await contract.methods.tokenURI(0).call();

console.log(String(total), owner, uri);
```

Expected output (three tokens minted so far):

output

```
3 0x5FbDB2315678afecb367f032d93F642f64180aa3 https://example.com/meta/0
```

`web3` returns big numbers as strings by default, so `String(total)` is only there to normalise older versions; never do arithmetic on `total` directly.

7.4 Common mistakes and best practices

Mistake	Why it hurts	Best practice
Writing your own ERC721 from scratch	easy to break transfer or approval semantics	extend OpenZeppelin's audited implementation
Forgetting `_safeMint` for contract recipients	NFTs can be locked in contracts that cannot manage them	use `_safeMint` unless the recipient is known to be an EOA
Expecting `tokenURI` to work without a base	a missing `_baseURI` returns an empty string	override `_baseURI` and set it in the constructor
Charging with `>=` instead of `==`	users overpay with no refund	check equality and document the price
Hard-coding the supply cap as a number in code	changing it later is impossible	`constant`, documented, and tested

7.5 Practice exercises

1. Add `function totalSupply() external view returns (uint256)` to `CritterNFT`.
2. Why is `critterName` not stored in contract storage in Example 1?
3. What does `safeTransferFrom` check that `transferFrom` does not?

Solutions

1.

Solidity

```
    function totalSupply() external view returns (uint256) {
        return nextId;            // ids minted so far; ownerMint also increments nextId
    }
```

Expected output: `3` after three mints of either kind.

2. Because a dynamic string per token costs two storage slots (length plus data) per token, and the name is only needed by the UI. It is emitted in the event and served in the token metadata instead.
3. That the recipient, if it is a contract, implements `onERC721Received`; if it does not, the transfer reverts instead of locking the token in a contract that cannot transfer it out.

7.6 Mini test

1. Which ERC functions let an operator move any token of a user?
2. True or false: ERC721 tokens are divisible into fractions.
3. What does `tokenURI(id)` return and where does the data actually live?
4. Which OpenZeppelin function creates a new token id?
5. Why must a minter check the return value of an ERC20 `transfer`?

Answer key and explanations

1. `setApprovalForAll(operator, true)` grants blanket permission; `approve(to, id)` grants it for a single token id.
2. False. Each id is a whole, unique item; fractional ownership needs a separate mechanism.
3. A URL (or base64 data URI) pointing to the token's metadata JSON, which lives off chain; the contract only stores the base URL.
4. `_safeMint(to, id)` (or `_mint`, without the receiver hook check).
5. Because some tokens do not revert on failure but return `false`, which the standard permits; a boolean check turns that into a hard failure.

Chapter 8 - A payable game economy

8.1 Summary

- `payable` marks a function (or the fallback) that may receive ether with the call; `msg.value` is the amount in wei.
- Ether held by a contract belongs to the contract, not to its deployer, and is only moved by functions that explicitly send it.
- Cooldowns compare `block.timestamp` with a stored deadline, so they measure elapsed time, not the clock's accuracy.
- On-chain randomness is unsafe: `block.timestamp`, `blockhash(block.number - 1)` and `block.prevrandao` are all visible to validators and miners, so games must not bet value on them.
- Serious randomness uses a commit-reveal scheme, a verifiable random function (VRF), or a future block oracle.
- `block.prevrandao` (formerly `block.difficulty`) is a beacon value that is not manipulable at will, but using it alone is still discouraged for high-value draws.
- Apply the checks-effects-interactions pattern in every payable function, as chapter 5 showed.
- Gas costs of storage make economy design part of security: charge enough to cover abuse.
- The winner-takes-all pattern (fee to a prize pool, payout by pull payment) keeps a game solvent even when one call fails.
- Randomness sources in practice: Chainlink VRF on mainnet and testnets, or a two-transaction commit-reveal on private networks.

8.2 Concepts

Randomness options compared:

Source	Manipulable	Recommended for
`block.timestamp`	yes (validator nudges)	nothing valuable
`blockhash(block.number - 1)`	yes	nothing valuable
`block.prevrandao`	partly (post-Merge)	low-stakes only
Commit-reveal by players	hard (needs collusion)	games where players reveal
Chainlink VRF (or similar)	no (verifiable)	anything with prizes

The fee-pool accounting:

output

```
battle fee per critic  -> prizePool += fee
winner resolved        -> prizePool -= payout (pull)
withdrawPayout()       -> sends the recorded amount once
```

Cooldown pattern:

Solidity

```
uint32 public constant COOLDOWN = 30 minutes;

function act() external {
    require(block.timestamp >= cooldownOf[msg.sender], "Critter: cooling down");
    cooldownOf[msg.sender] = uint32(block.timestamp) + COOLDOWN;
    // ... game logic
}
```

8.3 Examples

Example 1 - A cooldown-gated action with a fee

Solidity

```solidity
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.24;

/// @title Charge a fee, enforce a cooldown, keep the accounting pull-based
contract Arena {
    uint256 public constant ENTRY_FEE = 0.001 ether;
    uint32 public constant COOLDOWN = 30 minutes;

    mapping(address => uint32) public cooldownOf;
    mapping(address => uint256) public pendingWinnings;
    uint256 public prizePool;
    uint256 public battles;

    event Entered(address indexed player, uint256 pool);
    event Won(address indexed player, uint256 amount);
    event Withdrew(address indexed player, uint256 amount);

    error CoolingDown();
    error WrongFee();
    error NothingToWithdraw();

    function enter() external payable {
        if (msg.value != ENTRY_FEE) revert WrongFee();
        if (block.timestamp < cooldownOf[msg.sender]) revert CoolingDown();

        cooldownOf[msg.sender] = uint32(block.timestamp) + COOLDOWN;  // effect
        prizePool += msg.value;                                       // effect
        battles += 1;
        emit Entered(msg.sender, prizePool);
    }

    /// @dev Resolves a battle and credits the winner (pull payment)
    function resolve(address winner) external {
        uint256 prize = prizePool / 2;         // winner takes half the pool
        require(prize > 0, "Arena: empty pool");
        require(block.timestamp >= cooldownOf[winner], "Arena: winner cooling");

        prizePool -= prize;                    // effect first
        pendingWinnings[winner] += prize;
        emit Won(winner, prize);
    }

    function withdraw() external {
        uint256 amount = pendingWinnings[msg.sender];
        if (amount == 0) revert NothingToWithdraw();

        pendingWinnings[msg.sender] = 0;       // effect first, then interaction
        (bool sent, ) = msg.sender.call{value: amount}("");
        require(sent, "Arena: transfer failed");
        emit Withdrew(msg.sender, amount);
    }
}
```

Expected output (A enters, B enters, then `resolve(A)` is called):

output

```
enter() from A with 0.001 ETH  -> prizePool = 1e15 wei, battles = 1
enter() from A again           -> reverted: CoolingDown()
enter() from B with 0.0005 ETH -> reverted: WrongFee()
enter() from B with 0.001 ETH  -> prizePool = 2e15, battles = 2
resolve(A)                     -> pendingWinnings[A] = 1e15, prizePool = 1e15
withdraw() from A              -> A receives 1e15 wei, pendingWinnings[A] = 0
withdraw() from A again        -> reverted: NothingToWithdraw()
```

Example 2 - Commit-reveal randomness (sketch)

Solidity

```solidity
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.24;

/// @title Two-step randomness: commit a hash, reveal the secret later
contract Draw {
    struct Commitment {
        bytes32 hash;
        uint64 revealBy;
        bool revealed;
    }

    mapping(address => Commitment) public commits;
    bytes32[] public secrets;

    function commit(bytes32 hashedSecret) external {
        uint64 deadline = uint64(block.timestamp) + 1 hours;
        commits[msg.sender] = Commitment(hashedSecret, deadline, false);
    }

    function reveal(string calldata secret) external {
        Commitment storage c = commits[msg.sender];
        require(!c.revealed, "Draw: already revealed");
        require(block.timestamp <= c.revealBy, "Draw: too late");
        require(keccak256(abi.encodePacked(secret)) == c.hash, "Draw: mismatch");
        c.revealed = true;
        secrets.push(keccak256(abi.encodePacked(secret)));
    }

    /// @dev Call only after every player revealed
    function random() external view returns (uint256) {
        return uint256(keccak256(abi.encodePacked(secrets)));  // mix of revealed secrets
    }
}
```

Expected behaviour: two players commit, both reveal before the deadline, `random()` returns the same value for every caller because it derives from the stored secrets, and no single player can force the outcome, because the final mix depends on everyone's revealed value.

8.4 Common mistakes and best practices

Mistake	Why it hurts	Best practice
`block.timestamp` for a jackpot	validators can bias the draw by seconds	VRF or commit-reveal for anything with value

Mistake	Why it hurts	Best practice
Sending prizes inside the game loop	one reverting receiver blocks the round	credit balances, withdraw separately
Mixing cooldown check and update	an early-return leaves the old cooldown in place	set the deadline and effects only after checks pass
Accepting any value in a payable function	users overpay or underpay, accounting drifts	exact-fee checks, or refunds of the excess
Storing per-player state in unbounded loops	gas blows the block limit	aggregate counters, iterate only with clear bounds

8.5 Practice exercises

1. In `Arena`, why is the cooldown stored as `uint32`? What goes wrong with larger values?
2. Add a `refundUnused()` idea: instead, fix `enter` so excess ether is refunded while keeping the exact-fee accounting.
3. Why does `resolve` not transfer ether directly to the winner?

Solutions

1. `block.timestamp` fits an unsigned 32-bit number until the year 2106, so `uint32` packs into one slot. A comparison with a larger type would need a cast on both sides and still risk overflow arithmetic if you compute with it.

2.

Solidity

```solidity
    function enter() external payable {
        if (block.timestamp < cooldownOf[msg.sender]) revert CoolingDown();
        uint256 fee = msg.value;
        require(fee >= ENTRY_FEE, "Arena: too little");
        uint256 excess = fee - ENTRY_FEE;
        cooldownOf[msg.sender] = uint32(block.timestamp) + COOLDOWN;
        prizePool += ENTRY_FEE;
        battles += 1;
        if (excess > 0) {
            (bool sent, ) = msg.sender.call{value: excess}("");
            require(sent, "Arena: refund failed");
        }
        emit Entered(msg.sender, prizePool);
    }
```

Expected behaviour: `enter()` with 0.002 ETH takes 0.001 for the pool and refunds the rest, so no user can lock extra ether in the contract by accident.

3. Because a push payment inside `resolve` hands control to the winner's code during the resolution call: a reverting or reentrant receiver could block the whole round or re-enter `withdraw`. Crediting `pendingWinnings` and letting the winner withdraw keeps the effect separate from the interaction.

8.6 Mini test

1. Which variable holds the ether sent with a payable call, and in which unit?
2. True or false: `block.prevrandao` is fully safe for a casino jackpot.
3. What pattern prevents a reverting receiver from blocking payouts to everyone?
4. Which keyword makes a function able to receive ether?
5. What does checks-effects-interactions order mean for a payable function?

Answer key and explanations

1. `msg.value`, in wei (10^18 wei per ether).
2. False. It is influenced by the validator's randomness but not a strong, independent source; use a VRF for valuable draws.
3. Pull payments: record what each user may claim and let them withdraw.
4. `payable` (also usable on the `receive()` fallback for plain transfers).
5. Validate inputs, update storage, and only then call other contracts, so a reentrant or reverting call cannot corrupt the accounting.

Chapter 9 - Front-ends and Web3.js

9.1 Summary

- A DApp is a normal web page plus a wallet: the wallet holds keys and signs, the contract holds logic and data.
- MetaMask injects `window.ethereum` following EIP-1193, which exposes `request({method, params})` for every JSON-RPC call the page needs.
- `eth_requestAccounts` shows the connect prompt and returns the selected accounts; always call it from a user gesture.
- Reading state (a `view` call) is free and needs no signature; writing (a transaction) costs gas and needs the user to confirm in the wallet.
- A contract instance is built from its ABI and deployed address; the ABI is the compiler's JSON description of the functions and events.
- Sending a transaction returns a receipt once mined; before that you can listen for `transactionHash`.
- Events are subscribed with `contract.events.Name({fromBlock})` and delivered over a WebSocket or polling connection.
- The page must handle `chainChanged` and `accountsChanged`, because both can change at any time.
- MetaMask's default provider target is Ethereum mainnet; switch networks with `wallet_switchEthereumChain` and, for unknown networks, `wallet_addEthereumChain`.
- Ethers.js is the usual alternative to Web3.js, with a smaller bundle and a promise-based API; both talk to the same JSON-RPC.

9.2 Concepts

The DApp stack:

output

```
+------------------+        +------------------+        +-----------------+
|  Browser page    | wallet |  MetaMask        |  JSON- |  Ethereum node  |
|  (web3.js/ethers)| <----> |  (keys, signing) | <----> |  (state)        |
+------------------+        +------------------+        +-----------------+
         |                                                        ^
         | reads (eth_call) / sends (eth_sendRawTransaction)      |
         +--------------------------------------------------------+
```

Call versus transaction from the front-end:

	Read (`view` function)	**Write (transaction)**
Web3.js	`contract.methods.f(...).call()`	`contract.methods.f(...).send({from})`
Gas	none	paid by the signer
Wallet prompt	no	yes
Result	return value immediately	receipt after mining
Failure	`call` reverts with the reason	transaction reverts, gas is spent

The ABI is a JSON array of function descriptions. After `solc` compiles `Counter.sol`, the output contains the ABI and the bytecode; the front-end copies the ABI and, after deployment, the contract address.

9.3 Examples

Example 1 - Connecting a wallet and reading state

HTML

```
<!doctype html>
<html lang="en">
<head>
  <meta charset="utf-8">
  <title>Counter DApp</title>
</head>
<body>
  <h1>Counter</h1>
  <button id="connect">Connect wallet</button>
  <p>Account: <span id="account">not connected</span></p>
  <p>Count: <span id="count">-</span></p>
  <script type="module">
    import Web3 from 'https://esm.sh/web3@4';

    const abi = [
      { "inputs": [], "name": "count", "outputs": [{ "type": "uint256" }], "stateMutability": "view", "
      { "inputs": [], "name": "increment", "outputs": [], "stateMutability": "nonpayable", "type": "fun
    ];
    const address = '0x5FbDB2315678afecb367f032d93F642f64180aa3';

    let web3;
    let contract;

    async function connect() {
      if (!window.ethereum) throw new Error('MetaMask not found');
      web3 = new Web3(window.ethereum);
      const [account] = await window.ethereum.request({ method: 'eth_requestAccounts' });
      document.querySelector('#account').textContent = account;
      contract = new web3.eth.Contract(abi, address);
      refresh();
    }

    async function refresh() {
      const raw = await contract.methods.count().call();
      document.querySelector('#count').textContent = raw.toString();
    }

    document.querySelector('#connect').addEventListener('click', () => {
      connect().catch((error) => console.error(error));
    });
  </script>
</body>
</html>
```

Expected output in the browser after clicking Connect wallet and confirming in MetaMask:

output

```
Account: 0x742d35Cc6634C0532925a3b844Bc454e4438f44e
Count: 7
```

`count()` is a `view` function, so it runs locally through `eth_call` with no signature and no gas; the value is returned as a string because web3.js handles 256-bit integers as strings or BigInt.

Example 2 - Sending a transaction and watching events

JavaScript

```
// after connect() from Example 1
async function increment() {
  const [account] = await web3.eth.getAccounts();
  const receipt = await contract.methods.increment().send({ from: account });

  console.log('incremented in block', receipt.blockNumber.toString());
  console.log('gas used', receipt.gasUsed.toString());
  refresh();
}

contract.events.CountChanged({ fromBlock: 0 })
  .on('data', (event) => {
    console.log(`count is now ${event.returnValues.newCount} (changed by ${event.returnValues.by})`);
  })
  .on('error', (error) => console.error('event stream error', error));
```

Expected output in the browser console after one confirmed increment:

output

```
incremented in block 8612345
gas used 43218
count is now 8 (changed by 0x742d35Cc6634C0532925a3b844Bc454e4438f44e)
```

If the user rejects the transaction in MetaMask, the promise rejects with code `4001`; handling that error politely (no scary console trace) is part of good DApp UX.

9.4 Common mistakes and best practices

Mistake	Why it hurts	Best practice
Auto-calling `eth_requestAccounts` on page load	browsers block it without a gesture, and users get prompt fatigue	request accounts on a click
Assuming the account never changes	MetaMask can switch account or chain at any time	listen to `accountsChanged` and `chainChanged`, re-init state
Doing math on returned strings	web3 returns BigInt/strings, not `number`	use `BigInt(...)` arithmetic, format only for display
Hard-coding mainnet addresses	the same code breaks on testnets	keep per-network addresses in config
Ignoring failed transactions	users assume success and retry blindly	show the revert reason from the receipt and the gas spent

9.5 Practice exercises

1. Why does `contract.methods.count().call()` not open the wallet for confirmation?
2. What breaks if the user switches from Ethereum to Polygon while the page is open, and how do you fix it?
3. Write the web3.js snippet that adds an event listener printing only `ItemAdded` logs whose `author` is the connected account.

Solutions

1. Because `call` performs a local `eth_call` executed against the node's current state: it does not create a transaction, consumes no gas and changes nothing, so no signature and no wallet prompt are required.
2. The page keeps talking to the previous chain while the wallet signs for the new one, so transactions either fail or target addresses that do not exist on the other network. Fix it by handling `chainChanged`: reload or re-create the provider and reload the contract addresses for the new chain (MetaMask recommends `window.location.reload()` in the handler).
3.

JavaScript

```
const [me] = await web3.eth.getAccounts();
contract.events.ItemAdded({ fromBlock: 0, filter: { author: me } })
  .on('data', (event) => console.log(event.returnValues.label, event.returnValues.id));
```

Expected output: only events whose indexed `author` matches the connected account are delivered, one line per item, starting at block 0.

9.6 Mini test

1. Which MetaMask method opens the connect prompt and what does it return?
2. True or false: reading a `view` function from the browser costs gas.
3. Which two events signal that the wallet state changed under your page?
4. What is the ABI used for?
5. What are the two libraries commonly used for a DApp front-end?

Answer key and explanations

1. `eth_requestAccounts` (through `window.ethereum.request`); it returns the array of accounts the user approved.
2. False. Reads run as local `eth_call`s against a node, so only the node spends resources.
3. `accountsChanged` and `chainChanged` (both delivered through the same provider object).
4. It is the JSON description of the contract's functions and events that web3.js/ethers.js encode and decode with.
5. Web3.js and Ethers.js.

Chapter 10 - Tooling: Truffle, Ganache and deployment

10.1 Summary

- Truffle is the classic Solidity workflow tool: compile, migrate, test, and it wraps web3.js for scripts.

- `truffle init` scaffolds a project with `contracts/`, `migrations/`, `test/` and `truffle-config.js`.
- `truffle compile` runs the compiler, writes artifacts with the ABI and bytecode into `build/contracts/`.
- A migration is a JS file that deploys contracts in order, recording addresses per network in `build/contracts/*.json`.
- Ganache is a local in-memory blockchain with ten funded test accounts that makes deployment instant.
- `truffle-config.js` declares `networks` (host, port, network_id, gas) and the compilers' version.
- `truffle migrate --reset` redeploys everything; `--network <name>` picks the target from the config.
- Deployment to a public testnet needs a wallet: a `.env` mnemonic or a private key referenced from the config, never committed.
- Truffle Console (`truffle console`) gives a REPL with your deployed contracts loaded for manual calls.
- Hardhat is the modern alternative with a richer plugin ecosystem (hardhat-ethers, hardhat-verify); the concepts migrate directly.

10.2 Concepts

Project layout:

output

```
project/
  contracts/
    CritterNFT.sol        <- the source
  migrations/
    1_initial_migration.js
    2_deploy_critter.js   <- deployment scripts, in order
  test/
    critter.test.js       <- Mocha tests
  truffle-config.js       <- networks, compilers
  build/contracts/        <- compiled artifacts (ABI, bytecode, networks)
```

A migration script:

JavaScript

```
const CritterNFT = artifacts.require("CritterNFT");

module.exports = function (deployer) {
  deployer.deploy(CritterNFT, "https://example.com/meta/");
};
```

Artifacts are the handshake between Solidity and JavaScript: every artifact carries the contract's ABI, deployed address per network and the bytecode used to deploy it, and `artifacts.require("Name")` in any test or script resolves those fields.

10.3 Examples

Example 1 - A full local workflow

shell

```
mkdir critter-chain && cd critter-chain
npx truffle init
```

Expected output:

output

```
Starting init...
================

> Copying project files to ...

Init successful, sweet!
```

Put `CritterNFT.sol` in `contracts/` (from chapter 7) and write the migration:

JavaScript

```
const CritterNFT = artifacts.require("CritterNFT");

module.exports = function (deployer) {
  deployer.deploy(CritterNFT, "https://example.com/meta/");
};
```

Then deploy against Ganache (in a second terminal run `npx ganache` first, or keep the Ganache GUI running on port 7545):

shell

```
npx truffle compile
npx truffle migrate --network development --reset
```

Expected output (abridged):

output

```
Compiling your contracts...
===========================
> Compiling ./contracts/CritterNFT.sol
> Artifacts written to .../build/contracts
> Compiled successfully using:
   - solc: 0.8.24+commit.e11b9ed9.Emscripten.clang

Starting migrations...
======================
> Network name:    'development'
> Network id:      5777
> Block gas limit: 6721975

1_initial_migration.js
======================
   Replacing 'Migrations'
   ----------------------
   > transaction hash:    0x1f7...
   > Blocks: 0            Seconds: 0
   > contract address:    0xa1...
   > block number:        1
   > gas used:            222690

2_deploy_critter.js
===================
   Deploying 'CritterNFT'
   ----------------------
   > contract address:    0xb2...
   > gas used:            892455
   > Total cost:         0.0309 ETH
```

Example 2 - Deploying to a public testnet

JavaScript

```
// truffle-config.js
require('dotenv').config();            // npm install dotenv
const HDWalletProvider = require('@truffle/hdwallet-provider');

module.exports = {
  networks: {
    development: {
      host: "127.0.0.1",
      port: 7545,                      // Ganache GUI
      network_id: "*",
    },
    sepolia: {
      provider: () => new HDWalletProvider(process.env.MNEMONIC, "https://eth-sepolia.g.alchemy.com/v2/
      network_id: 11155111,
      gas: 5500000,
      confirmations: 2,
      timeoutBlocks: 200,
      skipDryRun: true
    }
  },
  compilers: {
    solc: { version: "0.8.24" }
  }
};
```

`.env` (never committed; add it to `.gitignore`):

output

```
MNEMONIC="twelve words from a throwaway wallet"
ALCHEMY_KEY="your alchemy key"
```

Expected result of `npx truffle migrate --network sepolia`:

output

```
2_deploy_critter.js
===================
   Deploying 'CritterNFT'
   ----------------------
   > contract address:    0xC5d2...
   > block number:        4731023
   > gas used:            902311
```

The address `0xC5d2...` is the live testnet deployment, which the front-end chapters reference; the same migration file deploys the same contract to every network, only the address changes.

10.4 Common mistakes and best practices

Mistake	Why it hurts	Best practice
Migrating without `--reset`	Truffle reuses the last recorded address and deploys nothing new	`--reset` when the contract changed
Committing `.env` with the mnemonic	full control of the wallet leaks to GitHub	`.env` in `.gitignore`, commit only `.env.example`
Using the same account everywhere	mainnet and testnet keys mix	a throwaway wallet per testnet
Deploying with the default low gas	migrations fail half-way and leave partial state	set `gas` per network, run `--dry-run` first
Editing a migration after it ran	the network's history no longer matches the script	write a new migration instead

10.5 Practice exercises

1. You changed only `CritterNFT.sol` and ran `truffle migrate` without flags; nothing new was deployed. Why?
2. What does `build/contracts/CritterNFT.json` contain that a front-end needs?
3. Write `truffle-config.js networks` entries for local development (Ganache GUI on 7545) and a Sepolia section using `@truffle/hdwallet-provider`.

Solutions

1. Truffle compares the migration scripts with what it already ran on that network; since no script changed, it prints "Network up to date" and skips deployment. Adding `--reset` re-runs every migration from scratch.

2. The ABI (for call encoding), the deployed address per network (`networks.<id>.address`), the bytecode, the compiler version and the source metadata - everything needed by web3.js/ethers.js or by a verification service.

3.

JavaScript

```
const HDWalletProvider = require('@truffle/hdwallet-provider');
require('dotenv').config();

module.exports = {
  networks: {
    development: { host: "127.0.0.1", port: 7545, network_id: "*" },
    sepolia: {
      provider: () => new HDWalletProvider(process.env.MNEMONIC, process.env.SEPOLIA_RPC_URL),
      network_id: 11155111,
      confirmations: 2,
      timeoutBlocks: 200,
      skipDryRun: true
    }
  },
  compilers: { solc: { version: "0.8.24" } }
};
```

Expected result: `npx truffle migrate --network sepolia` compiles (if needed), deploys and prints the contract address on Sepolia, while `--network development` targets Ganache.

10.6 Mini test

1. Which command scaffolds a Truffle project, and which folder holds migrations?
2. What does a migration file's `module.exports` function receive as its argument?
3. True or false: `truffle compile` deletes previously deployed addresses.
4. Which config key selects the Solidity compiler version in Truffle 5?
5. What must never appear in the repository when deploying to public networks?

Answer key and explanations

1. `truffle init`; migrations live in `migrations/`.
2. The `deployer` object, whose `deployer.deploy(Contract, ...args)` method queues deployments in order.
3. False. Artifacts keep network addresses; `compile` only refreshes the compiled output.
4. `compilers.solc.version` (in `truffle-config.js`).
5. Secrets: the mnemonic, private keys and RPC API keys - only `.env.example` with placeholder names belongs in git.

Chapter 11 - Testing smart contracts

11.1 Summary

- Contract tests run against a blockchain, not against mocks: Truffle deploys to Ganache and calls functions for real.
- `artifacts.require("Name")` gives the test a contract factory whose `.new()` deploys a fresh instance per test.
- Mocha's `describe`/`it` structure (Truffle aliases `contract`) groups cases and prints a readable report.
- Chai's `assert.equal` compares results, but every value from the chain is a string or BigInt, so convert before comparing.
- The receipt returned by a transaction contains `logs` and `events` you can assert on, which is how you test your own events.
- Every test gets the funded `accounts` array; pass `{from: accounts[0]}` to act as a specific wallet.
- Test failure paths too: every `require` and custom error deserves a test that expects the revert.
- Chai matchers from `chai`'s `revertedWith` add-on, or try/catch around `call`, are the two ways to assert reverts.
- `truffle test` runs everything in `test/`; `--network development` keeps the deployment persistent between runs when you skip Ganache's fresh state.
- Coverage (`truffle run coverage`) needs the solidity-coverage plugin and reports which lines never executed.

11.2 Concepts

Test anatomy:

output

```
contract("Arena", (accounts) => {
  let arena;
  beforeEach(async () => {            // fresh deployment per test
    arena = await Arena.new();
  });

  it("...", async () => {
    await arena.enter({ from: accounts[0], value: web3.utils.toWei("0.001") });
    const pool = await arena.prizePool();
    assert.equal(pool.toString(), "1000000000000000");
  });
});
```

Asserting events from a receipt:

output

```
const receipt = await instance.mint(...);
const event = receipt.logs[0];            // first (and only) log
assert.equal(event.event, "CritterMinted");
assert.equal(event.args.id.toString(), "3");
```

Receipt logs decode with the ABI automatically, which is why `event.args` fields are typed values rather than raw bytes.

11.3 Examples

Example 1 - A focused test suite

JavaScript

```
// test/arena.test.js
const Arena = artifacts.require("Arena");

contract("Arena", (accounts) => {
  const [owner, playerA, playerB] = accounts;
  const FEE = web3.utils.toWei("0.001", "ether");

  let arena;
  beforeEach(async () => {
    arena = await Arena.new({ from: owner });
  });

  it("collects the entry fee into the pool", async () => {
    await arena.enter({ from: playerA, value: FEE });
    const pool = await arena.prizePool();
    assert.equal(pool.toString(), FEE);
  });

  it("rejects the wrong fee", async () => {
    let reverted = false;
    try {
      await arena.enter({ from: playerB, value: web3.utils.toWei("0.001", "finney") });
    } catch (error) {
      reverted = error.message.includes("WrongFee");
    }
    assert.isTrue(reverted, "expected WrongFee revert");
  });

  it("enforces the cooldown", async () => {
    await arena.enter({ from: playerA, value: FEE });
    await arena.enter({ from: playerB, value: FEE });
    let reverted = false;
    try {
      await arena.enter({ from: playerA, value: FEE });
    } catch (error) {
      reverted = error.message.includes("CoolingDown");
    }
    assert.isTrue(reverted, "expected CoolingDown revert");
  });

  it("pays the winner through withdraw", async () => {
    await arena.enter({ from: playerA, value: FEE });
    await arena.enter({ from: playerB, value: FEE });
    await arena.resolve(playerA);

    const before = web3.utils.toBN(await web3.eth.getBalance(playerA));
    const receipt = await arena.withdraw({ from: playerA });
    receipt.logs.forEach(() => {});
    const gasCost = web3.utils.toBN(receipt.receipt.gasUsed).mul(web3.utils.toBN(1e9)); // rough
    const after = web3.utils.toBN(await web3.eth.getBalance(playerA));

    assert.equal(
      after.toString(),
      before.add(web3.utils.toBN(FEE).divn(2)).sub(gasCost).toString()
    );
  });
```

JavaScript (continued)

```
});
```

Expected output of `npx truffle test`:

output

```
Using network 'development'.

  Contract: Arena
    ✓ collects the entry fee into the pool (58ms)
    ✓ rejects the wrong fee (71ms)
    ✓ enforces the cooldown (84ms)
    ✓ pays the winner through withdraw (126ms)

  4 passing (1s)
```

Note the balance test: the player receives half the pool but also pays gas for the `withdraw` transaction, so the assertion is `before + prize - gasCost`. The `1e9` gas price above is a placeholder - use the receipt's `receipt.effectiveGasPrice` for exactness on modern clients.

Example 2 - Testing the NFT events

JavaScript

```
const CritterNFT = artifacts.require("CritterNFT");

contract("CritterNFT", (accounts) => {
  let nft;
  const PRICE = web3.utils.toWei("0.01", "ether");

  beforeEach(async () => {
    nft = await CritterNFT.new("https://example.com/meta/", { from: accounts[0] });
  });

  it("mints a token and emits CritterMinted", async () => {
    const receipt = await nft.mint("Fluffy", { from: accounts[1], value: PRICE });

    assert.equal(receipt.logs.length, 1, "one event");
    const event = receipt.logs[0];
    assert.equal(event.event, "CritterMinted");
    assert.equal(event.args.id.toString(), "0");
    assert.equal(event.args.to, accounts[1]);

    const owner = await nft.ownerOf(0);
    assert.equal(owner, accounts[1]);
  });

  it("rejects a wrong payment", async () => {
    let reverted = false;
    try {
      await nft.mint("Sparky", { from: accounts[1], value: 1 });
    } catch (error) {
      reverted = error.message.includes("WrongPayment");
    }
    assert.isTrue(reverted);
  });
});
```

Expected output:

output

```
Contract: CritterNFT
  ✓ mints a token and emits CritterMinted (102ms)
  ✓ rejects a wrong payment (65ms)

2 passing (890ms)
```

11.4 Common mistakes and best practices

Mistake	Why it hurts	Best practice
Comparing a returned string with a number	`"1" == 1` is true in JS but `assert.equal("1", 1)` can still surprise types	compare via `.toString()` or `toBN()`
Reusing one deployment across tests	order-dependent tests hide bugs	deploy fresh in `beforeEach`
Testing only the happy path	`require` bugs ship silently	one test per revert reason
Forgetting `{from}` for payable calls	the call comes from the default account and fails the fee check	always pass the acting account
Asserting on gasless float math	float drift makes balance checks flaky	use `web3.utils.toBN` for all on-chain values

11.5 Practice exercises

1. Write a test that asserts `nextId` equals `1` after one mint and `MAX_SUPPLY` reverts when minting the 10001st token.
2. How do you test that `transferOwnership` from a non-owner reverts?
3. Why is it wrong to assert `receipt.logs[0]` is your event without checking `logs.length`?

Solutions

1.

JavaScript

```
it("tracks ids and stops at MAX_SUPPLY", async () => {
  await nft.mint("One", { from: accounts[1], value: PRICE });
  assert.equal((await nft.nextId()).toString(), "1");

  let reverted = false;
  try {
    for (let i = 0; i < 10000; i += 1) {
      await nft.ownerMint(accounts[2]);
    }
    await nft.mint("Overflow", { from: accounts[1], value: PRICE });
  } catch (error) {
    reverted = error.message.includes("SoldOut");
  }
  assert.isTrue(reverted, "expected SoldOut revert");
});
```

Expected output: `console` shows the loop minting with `ownerMint` until `nextId` reaches 10000, then `mint` reverts with `SoldOut`. 2.

JavaScript

```
    it("keeps ownership safe", async () => {
      let reverted = false;
      try {
        await nft.transferOwnership(accounts[3], { from: accounts[1] });
      } catch (error) {
        reverted = error.message.includes("OwnableUnauthorizedAccount") ||
                   error.message.includes("NotOwner");
      }
      assert.isTrue(reverted);
    });
```

OpenZeppelin's `Ownable` (v5) reverts with the built-in `OwnableUnauthorizedAccount` custom error, so match both the custom-error name and any string reason your own modifiers use.

3. Because your expectation of exactly one event is the assertion: another log (or a missing one) shifts the array and silently tests the wrong thing. Assert both `logs.length` and the event name.

11.6 Mini test

1. Which artifact helper gives a test a deployable contract?
2. True or false: an address returned by `call()` can be compared with `===` to another address string.
3. What does the receipt returned by a transaction carry for event assertions?
4. Which web3 helper converts "0.001" strings to wei safely?
5. How do you test a `require` failure path in plain Mocha without extra plugins?

Answer key and explanations

1. `artifacts.require("ContractName")`, used as `await Contract.new(...)`.
2. True for equality comparisons of two lowercase hex strings from the same source; but never rely on checksummed formatting differences, and use `web3.utils.toChecksumAddress` when comparing user input.
3. `logs` (and `events` mapped by name); each log is decoded against the ABI, exposing `event.event` and `event.args`.
4. `web3.utils.toWei("0.001", "ether")`, which avoids floating-point loss.
5. Wrap the call in try/catch and assert that the promise rejected, preferably checking the revert reason in `error.message`.

Chapter 12 - Oracles and external data

12.1 Summary

- A contract cannot perform HTTP calls, because its execution must be deterministic for every node.
- An oracle is a bridge: an off-chain service reads external data and writes it on chain, where contracts can read it like any other state.
- The simplest oracle pattern is a data feed: a contract whose `latestRoundData()` returns the last price plus a freshness timestamp.
- Chainlink's `AggregatorV3Interface` is the reference implementation, with `decimals()` fixing the scale and `description()` naming the pair.
- Also available: Chainlink VRF for randomness (chapter 8) and "Functions"/request-fulfil oracles for arbitrary API calls.
- A request-fulfilment oracle needs two transactions: the contract emits a request, the oracle service calls back with the data.
- Trust is the core problem: feeds are decentralised across many nodes, while a single oracle is a trusted point of failure.
- Always check the feed's `updatedAt` freshness and scale by `10 ** decimals` before using a price.
- Reading an oracle from another contract is just an external `view` call wrapped in the feed's interface, so it is cheap and needs no transaction.
- Oracles are only as good as their aggregator: a halted feed must be handled (revert or fallback) in the consuming contract.

12.2 Concepts

The two architectures:

output

```
PUSH (data feed)
  many oracle nodes --> aggregator contract --> your contract reads latestRoundData()
  freshest data, continuous updates, gas paid by the oracle operator

PULL (request-fulfilment)
  your contract emits Request --> oracle service watches --> callback fulfils
  one request, one update, costs one callback transaction
```

Using a feed safely in your contract:

Solidity

```
interface AggregatorV3Interface {
    function decimals() external view returns (uint8);
    function description() external view returns (string memory);
    function latestRoundData() external view returns (
        uint80 roundId,
        int256 answer,
        uint256 startedAt,
        uint256 updatedAt,
        uint80 answeredInRound
    );
}
```

Your contract stores the feed address, calls `latestRoundData()`, and checks `updatedAt` is recent enough for its own rules.

12.3 Examples

Example 1 - Reading a Chainlink price feed

Solidity

```solidity
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.24;

import "@chainlink/contracts/src/v0.8/interfaces/AggregatorV3Interface.sol";

/// @title Charges a fixed USD price for a token sale, priced in ETH
contract UsdSale {
    AggregatorV3Interface internal immutable priceFeed;
    uint256 public constant MINT_PRICE_USD = 15 * 1e18;   // 15 dollars, 18 decimals
    uint256 public constant MAX_STALENESS = 3 hours;

    event Minted(address indexed to, uint256 ethPaid, uint256 ethUsdPrice);

    error PriceTooOld();
    error InsufficientPayment(uint256 sent, uint256 required);

    constructor(address feedAddress) {
        priceFeed = AggregatorV3Interface(feedAddress);
    }

    function latestPrice() public view returns (uint256 price, uint8 decimals_) {
        (, int256 answer, , uint256 updatedAt, ) = priceFeed.latestRoundData();
        if (block.timestamp - updatedAt > MAX_STALENESS) revert PriceTooOld();
        if (answer <= 0) revert PriceTooOld();
        price = uint256(answer);
        decimals_ = priceFeed.decimals();
    }

    function mint() external payable returns (uint256 ethDue) {
        (uint256 ethUsdPrice, uint8 decimals_) = latestPrice();
        // ethUsdPrice has 8 decimals (Chainlink USD feeds): USD(1e18) * 1e8 / price(1e8)
        ethDue = (MINT_PRICE_USD * (10 ** decimals_)) / ethUsdPrice;
        if (msg.value < ethDue) revert InsufficientPayment(msg.value, ethDue);
        emit Minted(msg.sender, msg.value, ethUsdPrice);
        // mint logic ...
    }
}
```

Expected behaviour when pointed at the Sepolia ETH/USD feed (`0x694AA1769357215DE4FAC081bf1f309aDC325306`; confirm the current address in the Chainlink docs):

output

```
decimals()  -> 8
description() -> "ETH / USD"
latestPrice() with ETH at $2,000 -> ethUsdPrice = 200000000000 (2000 * 1e8)
mint() sends 0.0075 ETH  -> ethDue = (15e18 * 1e8) / 200000000000 = 7500000000000000
                              Minted(buyer, 7.5e15, 2e11)
mint() sends 0.007 ETH   -> reverted: InsufficientPayment(7e15, 7.5e15)
feed not updated for 4 hours -> reverted: PriceTooOld()
```

Example 2 - A minimal request-fulfilment oracle

Solidity

```solidity
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.24;

/// @title Minimal oracle: request a number, a trusted service fulfils it
contract NumberOracle {
    address public immutable oracleService;
    uint256 public value;
    uint256 public lastUpdatedAt;

    event RequestNumber(uint256 indexed requestId, string query);
    event NumberFulfilled(uint256 indexed requestId, uint256 number);

    error NotOracle(address caller);

    constructor(address service) {
        oracleService = service;
    }

    function requestNumber(string calldata query) external returns (uint256 requestId) {
        requestId = uint256(keccak256(abi.encodePacked(block.timestamp, msg.sender, value)));
        emit RequestNumber(requestId, query);
    }

    function fulfil(uint256 requestId, uint256 number) external {
        if (msg.sender != oracleService) revert NotOracle(msg.sender);
        value = number;
        lastUpdatedAt = block.timestamp;
        emit NumberFulfilled(requestId, number);
    }
}
```

Matching node-side listener (shape of the off-chain service):

JavaScript

```javascript
const Web3 = require('web3');
const web3 = new Web3(process.env.RPC_URL);
const oracle = new web3.eth.Contract(oracleAbi, oracleAddress);
const key = web3.eth.accounts.privateKeyToAccount(process.env.ORACLE_KEY).address;

oracle.events.RequestNumber({ fromBlock: 'latest' })
  .on('data', async (event) => {
    const number = await fetchExternalValue();      // your HTTP call here
    await oracle.methods
      .fulfil(event.returnValues.requestId, String(number))
      .send({ from: key, gas: 200000 });
  });
```

Expected behaviour: `requestNumber("random")` logs `RequestNumber(0x3f..., "random")`, the service's listener wakes, and `NumberFulfilled(0x3f..., 42)` follows a few seconds later; `value` is then readable by any contract that knows this oracle's address.

12.4 Common mistakes and best practices

Mistake	Why it hurts	Best practice
Trusting `answer` without checking `updatedAt`	a halted feed serves stale prices	revert after your own staleness bound
Mixing feed decimals	8-decimal prices multiplied naively are off by 10^10	always scale with `10 ** decimals()`
Pointing at a single oracle node	one outage or lie corrupts the contract	use decentralised feeds or multiple nodes
Ignoring a negative or zero `answer`	some feeds return sentinel values	revert on `answer <= 0`
Calling `latestRoundData` inside a loop	many RPC calls, many failure points	read once, reuse the value

12.5 Practice exercises

1. In `UsdSale`, what does the consumer lose if `updatedAt` is 55 minutes old and `MAX_STALENESS` is 3 hours?
2. Why is the request-fulfilment pattern not usable for a same-block price?
3. What is the purpose of `decimals()` on a Chainlink feed, and why must your math respect it?

Solutions

1. Nothing is enforced by the contract, but the price carries risk: an hourly update means the value is fresh enough for a 3-hour window by contract rule, yet the sale accepts whatever the feed last published, so during volatile markets users could over- or under-pay relative to spot.
2. Because the fulfil step is a second transaction that only exists after the oracle service has observed the request and mined its own callback, which necessarily lands in a later block.
3. It states the number of decimal places in `answer` (usually 8 for USD prices). If your contract treats the value as 18-decimal currency, every derived amount is off by a factor of 10^10, which is why the price must be scaled with `10 ** decimals()` before dividing.

12.6 Mini test

1. Why can a contract perform no HTTP GET?
2. Which Chainlink function returns the current answer and the round it belongs to?
3. True or false: a data feed updates when your contract asks for it.
4. What must you check besides the price value before trusting a feed?
5. Which two patterns deliver external data to a contract?

Answer key and explanations

1. Because execution must be deterministic for every node, and network answers could differ per node and per time.

2. `latestRoundData()`, which returns the round id, the answer and the timestamps.
3. False. The oracle operator pays for updates continuously; your contract only reads the latest stored value.
4. `updatedAt` (freshness) and that `answer` is positive; treat a stale feed as unavailable.
5. Push feeds (updated by many oracle nodes) and pull request-fulfilment oracles (one request, one callback).

Chapter 13 - The L2 and multi-chain landscape

13.1 Summary

- Layer 2 (L2) networks sit on top of Ethereum: they execute transactions elsewhere and publish the result back to L1 for security.
- Optimistic rollups post a fraud-proof window: anyone can challenge an invalid state root within roughly seven days.
- ZK rollups post validity proofs: every batch is mathematically verified on L1, so withdrawals wait for no challenge period.
- Both families are EVM-compatible, so the same Solidity and the same tools mostly work, with a few opcodes and gas rules differing.
- zkSync and StarkNet add their own system contracts and account abstraction, so the developer flow differs from vanilla Ethereum.
- Sidechains (Polygon PoS, BNB Chain) have their own consensus and are not L2s in the strict sense; security depends on their validators, not on Ethereum.
- Other ecosystems reuse Solidity differently: TRON uses a TVM with TRC-20/TRC-721, Neo uses C#, Polkadot parachains use ink! (Rust).
- Bridging assets between chains happens through bridge contracts that lock on one side and mint on the other.
- Deployment is the same shape everywhere: compile once, deploy per network, keep an address per chain in your front-end config.
- Gas tokens differ per chain (ETH, MATIC/POL, BNB, TRX), so "gas price" is not comparable across networks without normalisation.

13.2 Concepts

Scaling families compared:

Family	Security from	Finality	Examples
Rollup (optimistic)	Ethereum data + fraud proofs	about 7 days for withdrawals	Optimism, Arbitrum, Base

Family	Security from	Finality	Examples
Rollup (ZK)	Ethereum data + validity proofs	minutes to hours	zkSync Era, StarkNet, Scroll, Polygon zkEVM
Sidechain	its own validators	chain finality	Polygon PoS, BNB Chain
L1	its own consensus	own finality	Ethereum, Avalanche, TRON, Neo

What stays the same on EVM chains: the Solidity language, the EVM opcodes (with exceptions), the ABI, the tooling (Hardhat, Truffle, Foundry) and the front-end libraries. What changes: RPC URLs, chain ids, gas accounting, precompiles and system contracts like the entry point for account abstraction.

13.3 Examples

Example 1 - Chain-aware configuration in one file

JavaScript

```
// config/networks.js
module.exports = {
  1: { name: "Ethereum", rpc: process.env.MAINNET_RPC, explorer: "https://etherscan.io", currency: "ETH
  11155111: { name: "Sepolia", rpc: process.env.SEPOLIA_RPC, explorer: "https://sepolia.etherscan.io",
  10: { name: "Optimism", rpc: process.env.OP_RPC, explorer: "https://optimistic.etherscan.io", currenc
  42161: { name: "Arbitrum One", rpc: process.env.ARB_RPC, explorer: "https://arbiscan.io", currency: "
  324: { name: "zkSync Era", rpc: process.env.ZKSYNC_RPC, explorer: "https://explorer.zksync.io", curre
};
```

Expected output when the front-end initialises on a local Ganache (`network_id 5777`, unknown to the table):

output

```
Unknown network 5777: switching to Sepolia by default
Loaded CritterNFT at 0xC5d2... on Sepolia
```

The fallback keeps the app usable instead of showing a broken page, and the same contract ABIs work everywhere because all these chains speak the EVM.

Example 2 - Detecting the chain in a contract

Solidity

```solidity
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.24;

/// @title Behaves differently on zkSync Era than on Ethereum mainnet
contract ChainAware {
    uint256 public immutable chainId;

    constructor() {
        chainId = block.chainid;         // 1 mainnet, 324 zkSync Era, 11155111 Sepolia
    }

    function isZkSync() external view returns (bool) {
        return chainId == 324;
    }

    /// @dev L2s batch proofs; use a longer finality margin there
    function finalityMargin() external view returns (uint256 seconds_) {
        seconds_ = isZkSync() ? 10 minutes : 15 minutes;
    }
}
```

Expected output:

output

```
chainId()        -> 324 (deployed on zkSync Era)
isZkSync()       -> true
finalityMargin() -> 600
```

`block.chainid` is set by the chain, not the transaction, so the value is fixed for a deployment; reading it lets one codebase adapt its rules per network instead of forking per network.

13.4 Common mistakes and best practices

Mistake	Why it hurts	Best practice
Assuming all chains finalise together	back-running before finality drains bridges and games	per-chain finality margins
Treating L2 fees as negligible	proof and calibration fees can add up on small batches	gate actions by the actual `eth_estimateGas` result
Deploying to a sidechain and calling it "L2"	security claims mislead users	document the trust model explicitly
Bridging without verifying the target	a wrong chain id mints on the wrong network	read the chain id from the wallet, not from the config
Hard-coding one explorer URL	users on other chains get broken links	keep per-chain explorer base URLs

13.5 Practice exercises

1. Why do optimistic rollups force a waiting period before withdrawals, while ZK rollups do not?
2. A contract works on Ethereum mainnet but reverts on an L2. Name two reasons.

3. What does the front-end need in order to offer the same contract on three chains?

Solutions

1. Because an optimistic rollup's state root is only final once no fraud proof succeeded during the challenge window; ZK rollups publish a validity proof for each batch, so L1 verifies the transition before accepting it and no waiting is needed.
2. Common causes: a precompile or system contract that does not exist on the L2, and gas accounting differences (for example the 64/63 rule and the 50M gas-per-batch cap on some ZK rollups) that make a transaction exceed the batch limits.
3. The ABI plus one deployed address per chain, the chain id of each target, and an RPC endpoint for each network, all stored in a per-chain config; plus detection of which chain the connected wallet is on so it can prompt to switch.

13.6 Mini test

1. Which two rollup families exist and how does each prove correctness to L1?
2. True or false: sidechains such as Polygon PoS inherit Ethereum's security.
3. Which field reveals the chain a transaction executes on?
4. Why do withdrawals from an optimistic rollup wait about seven days?
5. Which language do Neo N3 contracts use?

Answer key and explanations

1. Optimistic rollups (assume valid, prove invalid via fraud proofs) and ZK rollups (prove validity with zero-knowledge proofs published on L1).
2. False. A sidechain's security comes from its own validators, though it settles bridges and checkpointing back to Ethereum.
3. `block.chainid`, available in any contract.
4. The challenge window: during it anyone can submit a fraud proof that invalidates the root, so finality is only guaranteed once it elapses.
5. C# (with .NET tooling), via Neo's contract framework.

Chapter 14 - Capstone: critter-battle DApp

14.1 Summary

- A complete DApp is three artefacts: one Solidity contract set, a test suite, and a front-end that reads and writes through a wallet.
- The battle contract of chapters 2-8 gains an ERC721 identity layer, so winning changes which token the player holds.
- The project separates concerns: `CritterNFT` owns tokens and metadata, `Arena` owns fees, cooldowns and payouts, and the front-end only knows both ABIs.

- Deployment is reproducible through one migration file per network and a per-network address table used by the front-end.
- Tests cover the three layers: unit tests for the contract rules, event tests for the UI contract, and a manual pass through the browser flow.
- The safest flow for the front-end is read on load, write on click, and refresh from the receipt.
- Randomness stays off the contract: a commit-reveal draw or a VRF feeds the result, and the contract only verifies it.
- Fees stay in the contract as a prize pool, credited by battle and claimed by pull payment.
- The owner keys live in a wallet the project never sees: only the mnemonic in `.env` signs deployment.
- Before mainnet the checklist below must be green: validation, payout rules, event coverage and a fresh audit by a second pair of eyes.

14.2 Concepts

Capstone architecture:

output

```
                     +--------------------+
    owner (multisig) | CritterNFT (721)   |  mint, metadata, transfer
                     +---------+----------+
                               | ownerOf / safeTransferFrom
                     +---------v----------+
    players          | Arena              |  fee, cooldown, prize pool
                     +----+----------+----+
                          |          | withdraw() pull
              prizePool credit     event Won
                          |          |
                     +----v----------v----+
    front-end        | web3.js + MetaMask | read, send, subscribe
                     +--------------------+
```

Ownership boundary: `Arena` never stores tokens; it asks `CritterNFT.ownerOf(id)` and records battle results keyed by token id. That keeps the token contract simple and lets the battle rules be replaced independently (a new `Arena` address) without touching the collectibles.

Data flow of one battle:

output

```
battle(attackerId, defenderId)
  1. check: both tokens exist, both off cooldown, caller owns the attacker
  2. check: random server verified (VRF or commit-reveal) in a previous tx
  3. effect: cooldowns set, pool fee collected, winner resolved
  4. event: BattleResult(winnerId, loserId, feePaid)
  5. front-end: refresh owned tokens and balances
```

14.3 Examples

Example 1 - Battle settlement (the heart of the DApp)

Solidity

```solidity
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.24;

import "@openzeppelin/contracts/token/ERC721/IERC721.sol";

/// @title Resolves token battles with fees and cooldowns, no randomness on chain
contract Arena {
    IERC721 public immutable critters;
    uint256 public constant BATTLE_FEE = 0.0005 ether;
    uint32 public constant COOLDOWN = 1 minutes;

    mapping(uint256 => uint32) public cooldownOf;
    mapping(address => uint256) public claimable;
    uint256 public prizePool;
    uint256 public battlesFought;

    struct Battle {
        uint256 attackerId;
        uint256 defenderId;
        uint256 seed;          // verified randomness supplied off chain
    }

    event BattleSettled(uint256 indexed attackerId, uint256 indexed defenderId,
                        uint256 indexed winnerId, address caller, uint256 fee);
    event Withdrew(address indexed who, uint256 amount);

    error NotCritterOwner(uint256 tokenId);
    error CoolingDown(uint256 tokenId);
    error BadSeed();
    error NothingToWithdraw();

    constructor(address nftAddress) {
        critters = IERC721(nftAddress);
    }

    function battle(Battle memory b) external payable {
        if (b.seed == 0) revert BadSeed();
        if (msg.value != BATTLE_FEE) revert BadSeed();
        if (critters.ownerOf(b.attackerId) != msg.sender) revert NotCritterOwner(b.attackerId);
        if (critters.ownerOf(b.defenderId) == address(0)) revert NotCritterOwner(b.defenderId);
        if (cooldownOf[b.attackerId] > block.timestamp) revert CoolingDown(b.attackerId);
        if (cooldownOf[b.defenderId] > block.timestamp) revert CoolingDown(b.defenderId);

        // effects before any external interaction
        cooldownOf[b.attackerId] = uint32(block.timestamp) + COOLDOWN;
        cooldownOf[b.defenderId] = uint32(block.timestamp) + COOLDOWN;
        prizePool += BATTLE_FEE;
        battlesFought += 1;

        // settlement: attacker wins when the low bit of the verified seed is 0
        uint256 winnerId = (b.seed % 2 == 0) ? b.attackerId : b.defenderId;
        uint256 reward = BATTLE_FEE / 2;
        claimable[critters.ownerOf(winnerId)] += reward;   // credited to the winner's owner

        emit BattleSettled(b.attackerId, b.defenderId, winnerId, msg.sender, BATTLE_FEE);
    }
```

Solidity (continued)

```
57
58      function withdraw() external {
59          uint256 amount = claimable[msg.sender];
60          if (amount == 0) revert NothingToWithdraw();
61          claimable[msg.sender] = 0;
62          (bool sent, ) = msg.sender.call{value: amount}("");
63          require(sent, "Arena: transfer failed");
64          emit Withdrew(msg.sender, amount);
65      }
66  }
```

Expected behaviour (tokens 0 and 1 owned by A and B):

output

```
battle({attackerId: 0, defenderId: 1, seed: 42}) from A with 0.0005 ETH
   -> seed even, so token 0 wins; claimable[A] = 0.00025 ETH
   -> BattleSettled(0, 1, 0, A, 0.0005ether)
battle(...) from A again immediately -> reverted: CoolingDown(0)
battle({... seed: 43}) from A        -> token 1 wins; claimable[B] += 0.00025 ether
withdraw() from A                    -> A receives 0.00025 ETH
withdraw() from B after a win        -> B receives the credit
```

Example 2 - Front-end flow for the capstone

JavaScript

```
async function loadArena(account) {
  const arena = new web3.eth.Contract(arenaAbi, arenaAddress);
  const [pool, battles] = await Promise.all([
    arena.methods.prizePool().call(),
    arena.methods.battlesFought().call()
  ]);
  render(`${web3.utils.fromWei(pool, 'ether')} ETH across ${battles} battles`);
}

async function submitBattle(attackerId, defenderId, seed) {
  const [account] = await web3.eth.getAccounts();
  await arena.methods
    .battle({ attackerId, defenderId, seed })
    .send({ from: account, value: '500000000000000' });       // 0.0005 ETH
  await loadArena(account);                                     // refresh from chain
}

arena.events.BattleSettled({ fromBlock: 0 })
  .on('data', (event) => {
    log(`token ${event.returnValues.winnerId} beat ${event.returnValues.defenderId}`);
  });
```

Expected output in the page after one battle:

output

```
0.0005 ETH across 1 battles
token 0 beat 1
```

Example 3 - Release checklist for the capstone

output

```
[ ] truffle compile          no warnings about unused variables
[ ] truffle test             100% of require branches covered
[ ] battle cooldowns         reverts on second call within COOLDOWN
[ ] prize withdrawal         payouts equal entries minus the 50% reward
[ ] front-end read/write     works on Sepolia with a funded test wallet
[ ] events                   BattleSettled and Withdrove readable on the explorer
[ ] owner addresses          multisig on testnet, EOA only for experiments
[ ] addresses table          one entry per network in config/networks.js
```

14.4 Common mistakes and best practices

Mistake	Why it hurts	Best practice
Passing `seed` from the front-end	the player selects the winner in clear text	verify randomness on chain (VRF or commit-reveal)
Crediting the winner by caller instead of token owner	a proxy caller steals the reward	use `critters.ownerOf(winnerId)`
Charging fees inside the battle resolution	the interaction happens after the effects	collect fees first, settle effects, no calls out
Front-end guessing balances	stale UI undermines trust	refresh after the receipt lands
One giant contract for everything	a token bug loses the game, and vice versa	separate token and game contracts

14.5 Practice exercises

1. Add `function retreat(uint256 tokenId) external` that ends the battle early, halves the caller's chance of reward by crediting the fee to the pool only, and explains the check needed.
2. Why must `battle` check `critters.ownerOf(b.defenderId) != address(0)`?
3. The front-end passes a `seed` from a button. What is the security problem, and which chapter's mechanism fixes it?

Solutions

1.

Solidity

```solidity
    function retreat(uint256 tokenId) external {
        if (critters.ownerOf(tokenId) != msg.sender) revert NotCritterOwner(tokenId);
        if (cooldownOf[tokenId] > block.timestamp) revert CoolingDown(tokenId);
        cooldownOf[tokenId] = uint32(block.timestamp) + COOLDOWN;
        // fee stays in the pool; no reward credit is given
    }
```

The needed check is ownership: only the token's owner may retire it from a battle, so the function reverts with `NotCritterOwner` otherwise.

2. Because `ownerOf` for a burned or never-minted id returns `address(0)`, and allowing it would let anyone "battle" a phantom token and receive a reward.
3. The player can choose the seed that makes their token win, so the outcome is a guess the contract cannot verify. The fix is chapter 8's verified randomness (commit-reveal or a VRF callback) so the contract only accepts seeds it can check.

14.6 Mini test

1. Which three artefacts make the capstone complete?
2. Why does `battle` set cooldowns before emitting the result event?
3. True or false: the Arena contract may store NFT metadata.
4. What is the pull-payment pattern for the prize pool?
5. Which chapter-5 rule governs every external call in `battle`?

Answer key and explanations

1. The contract set (NFT + arena), the test suite, and the front-end with wallet wiring.
2. Because the state effects must happen first: if the function reverted after emitting, or if the event arrived before the storage updates, the front-end would display a battle whose cooldowns never applied.
3. False. It only calls `ownerOf`; metadata and minting belong to the token contract.
4. Credit `claimable[owner]` during settlement, then let the owner withdraw through a separate transaction.
5. Checks-effects-interactions: verify conditions, update storage, and only then interact - here the only interactions are the `ownerOf` reads and the `withdraw` payment.

Cheat sheet

15.1 Language basics

Item	Syntax
File header	`// SPDX-License-Identifier: MIT`
Pragma	`pragma solidity ^0.8.24;`
Contract	`contract Name is Base { ... }`
Constructor	`constructor(...) Base(...) { ... }`
State var	`uint256 public count;` / `address immutable owner;`
Constant	`uint256 public constant MAX = 100;`
Struct	`struct S { string name; uint32 level; }`
Mapping	`mapping(address => uint256) public balances;`

Item	Syntax
Array	`S[] items; S[4] fixed;`
Event	`event Done(uint256 indexed id);`
Custom error	`error NotOwner(address caller);` + `revert NotOwner(msg.sender)`

15.2 Visibility and mutability

Keyword	Meaning
`private` / `internal`	this contract / contract plus inheritors
`public` / `external`	everywhere / outside only
`view`	reads state, no writes
`pure`	no state access
`payable`	may receive ether

15.3 Units and constants

Value	Equals
`1 wei`	1
`1 gwei`	1e9 wei
`1 ether`	1e18 wei
`1 seconds` / `minutes`	1 / 60
`msg.value`	wei sent with the call
`block.timestamp`	block time in seconds
`block.chainid`	network id

15.4 Gas quick reference

Operation	Gas
Cold storage write	about 20,000
Warm storage write	2,900
CALLLOAD (calldata bytes)	16 per non-zero byte, 4 per zero byte
Event base (2 topics)	about 1,500
External call (base)	2,600 + execution

15.5 Battle-hardened snippets

Solidity

```solidity
// owner check
if (msg.sender != owner) revert NotOwner(msg.sender);

// safe ether send (after effects)
(bool sent, ) = to.call{value: amount}("");
require(sent, "transfer failed");

// reentrancy guard idea
require(!locked, "reentrant"); locked = true; _; locked = false;

// commit-reveal check
require(keccak256(abi.encodePacked(secret)) == hash, "mismatch");
```

15.6 Tooling commands

Task	Command
Local chain	`npx ganache`
Init project	`npx truffle init`
Compile	`npx truffle compile`
Test	`npx truffle test`
Deploy local	`npx truffle migrate --network development --reset`
Deploy testnet	`npx truffle migrate --network sepolia`
Console	`npx truffle console --network sepolia`

15.7 Front-end essentials

JavaScript

```javascript
const web3 = new Web3(window.ethereum);
await window.ethereum.request({ method: 'eth_requestAccounts' });   // connect
await web3.eth.getChainId();                                        // network id
const c = new web3.eth.Contract(abi, address);
await c.methods.count().call();                                     // read
await c.methods.increment().send({ from: account });               // write
c.events.Done({ fromBlock: 0 }).on('data', (e) => console.log(e));  // events
```

Final test

1. Which line must be the first statement in a modern Solidity file?
2. What is the difference between `memory` and `storage`?
3. True or false: `uint8` addition wraps silently since Solidity 0.8.

4. Which function visibility keyword allows calls only from outside the contract?
5. What does `msg.value` contain?
6. Which statement ordering defeats reentrancy?
7. What is the maximum number of `indexed` event parameters?
8. True or false: a contract can perform an HTTP GET inside a transaction.
9. What does `mapping(address => uint256)` NOT let you do?
10. Which keyword marks a base function as overridable?
11. What problem does the ERC721 `approve` function solve?
12. What does `tokenURI(id)` return?
13. Which mechanism makes on-chain randomness safe for prizes?
14. A payable function receives 2x the required fee. What should it do?
15. Which Truffle command deploys contracts in `migrations/` order to the configured network?
16. What does an artifact JSON file provide to a front-end?
17. How does a Mocha test deploy a contract under test?
18. What makes a Chainlink feed value unusable even if the price looks correct?
19. How do optimistic rollups differ from ZK rollups in proving correctness?
20. Which field tells a contract which chain it lives on?
21. Write a function that returns whether the caller is the stored owner.
22. Write the line that sends all of `amount` wei to `msg.sender` after zeroing the balance.
23. An NFT marketplace needs to move tokens for a user. Which two functions enable that?
24. A user's `withdraw` transaction reverts; their balance was already set to zero. What does that mean for the contract?
25. Why must `receive()` update balances instead of just accepting ether?

16.1 Final test answers

1. The licence identifier `// SPDX-License-Identifier: MIT`, followed by `pragma solidity ^0.8.24;`.
2. `storage` is the permanent contract state and survives between calls; `memory` is a temporary buffer released when the call ends.
3. False. Overflow reverts the transaction; wrapping needs `unchecked`.
4. `external`.
5. The amount of ether attached to the call, in wei.
6. Checks-effects-interactions: validate inputs, update storage, then call other contracts.
7. Three, because each indexed parameter becomes a log topic and a log has at most four topics (the first is the event signature).
8. False. Network access would break determinism; data must arrive through an oracle.
9. Enumerate its keys or read its length: you must track key lists separately.
10. `virtual` (paired with `override` in the derived contract).

11. It lets a third party, such as a marketplace, transfer a specific token on the owner's behalf without full custody of all tokens.
12. A URL (or base64 data URI) to JSON metadata describing that token: name, description, image and attributes.
13. A source the contract can verify: Chainlink VRF, or a commit-reveal scheme mixing everyone's revealed secrets.
14. Either reject any value other than the exact fee, or refund the excess immediately - never keep the surplus silently.
15. `truffle migrate` (optionally with `--network <name>` and `--reset`).
16. The ABI, the bytecode, the compiler version and the deployed address per network: everything needed to encode calls and locate the deployment.
17. With `const C = artifacts.require("C"); const instance = await C.new(...args);`, which deploys a fresh instance against the test network.
18. Staleness: `updatedAt` too old (or a non-positive answer) means the value no longer reflects the market, so the consumer must reject it.
19. Optimistic rollups assume validity and let anyone challenge with a fraud proof; ZK rollups attach a validity proof to every batch, so L1 verification alone settles it.
20. `block.chainid`.
21.

Solidity

```
    function isCallerOwner() external view returns (bool) {
        return msg.sender == owner;
    }
```

22.

Solidity

```
    balances[msg.sender] = 0;
    (bool sent, ) = msg.sender.call{value: amount}("");
    require(sent, "transfer failed");
```

The balance must be zeroed before the external call.

23. `approve(operator, tokenId)` for one token and `setApprovalForAll(operator, true)` for every token the owner holds.
24. The call reverted atomically, so both the zeroed balance and the failed transfer rolled back: the user keeps their balance and no ether left the contract.
25. Because a plain ether transfer to a contract with `payable receive()` would otherwise be uncredited ether with no accounting, and users could never withdraw it; the rule "every incoming wei is recorded" keeps the ledger sound.

Sources

- https://cryptozombies.io/en/solidity - CryptoZombies Solidity course (lesson order for chapters 2-8)
- https://cryptozombies.io/en/course - CryptoZombies advanced courses (Truffle, oracles, oracles part 2-3, zkSync, TRON, Neo)
- https://docs.soliditylang.org/en/v0.8.24/ - Solidity language reference (types, visibility, events, inheritance, security considerations)
- https://docs.soliditylang.org/en/v0.8.24/security-considerations.html - Security considerations (reentrancy, randomness caveats)
- https://docs.soliditylang.org/en/v0.8.24/units-and-global-variables.html - Units and global variables (wei, block.timestamp, msg.sender)
- https://ethereum.org/en/developers/docs/ - Ethereum developer documentation (accounts, transactions, gas)
- https://ethereum.org/en/developers/docs/evm/ - Ethereum Virtual Machine overview
- https://ethereum.org/en/developers/docs/standards/tokens/ - Token standards overview
- https://eips.ethereum.org/EIPS/eip-721 - ERC721 non-fungible token standard
- https://eips.ethereum.org/EIPS/eip-20 - ERC20 token standard
- https://docs.openzeppelin.com/contracts/5.x/ - OpenZeppelin Contracts 5.x reference (ERC721, Ownable, ReentrancyGuard)
- https://docs.chain.link/data-feeds - Chainlink Data Feeds (AggregatorV3Interface usage and feed addresses)
- https://docs.chain.link/vrf - Chainlink VRF verifiable randomness
- https://trufflesuite.com/docs/ - Truffle documentation (projects, migrations, testing)
- https://hardhat.org/hardhat-runner/docs - Hardhat documentation (workflow, plugins, verification)
- https://docs.zksync.io/ - zkSync Era developer documentation
- https://remix.ethereum.org/ - Remix browser IDE for quick experiments